



G1, ZGC, Shenandoah, ...

Chez Java, ils sont gentils avec tous
leurs GCs mais moi je choisis lequel ?



Antoine DESSAIGNE

Ce qu'on va voir ensemble

Comprendre les GCs (Garbage Collectors) (en Java)

Qui suis-je ?



Antoine DESSAIGNE
architecte logiciel

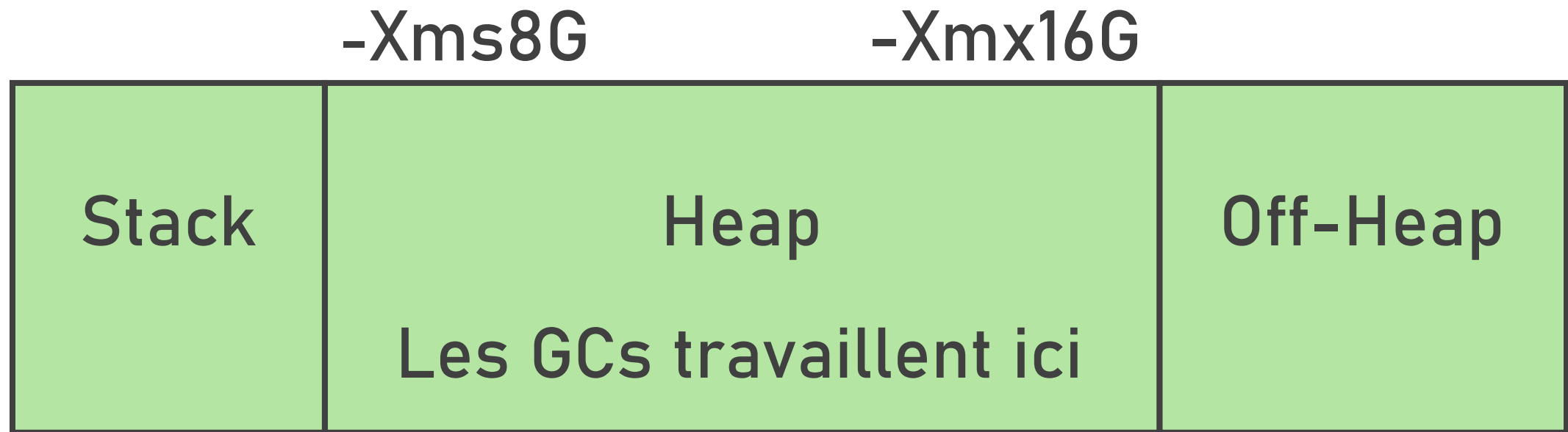


Éditeur français de logiciel
environ 4800 employés

A quoi sert un GC ?

- Gérer la mémoire à notre place
 - allocation de mémoire (malloc)
 - libération de mémoire (free)
- pour éviter les memory leaks
- pour éviter d'écrire au mauvais endroit

La mémoire dans Java



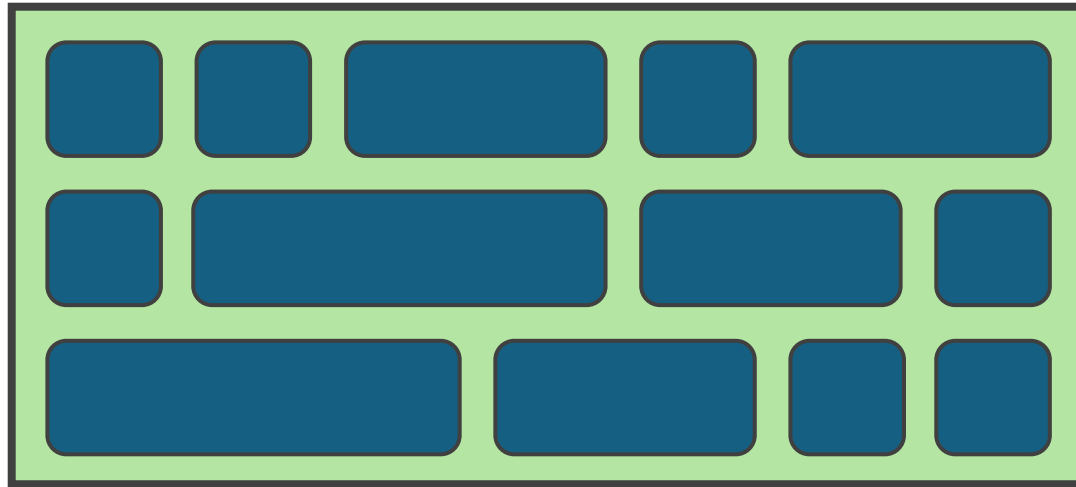
Au commencement...

Java 1

- Il y a 30 ans, en 1996
- Conçu pour l'électronique grand public
- Un ordinateur de 1996
 - CPU : 100 à 166 Mhz
 - RAM : 8 à 16 Mo
 - Stockage : 500 à 1200 Mo



Java 1



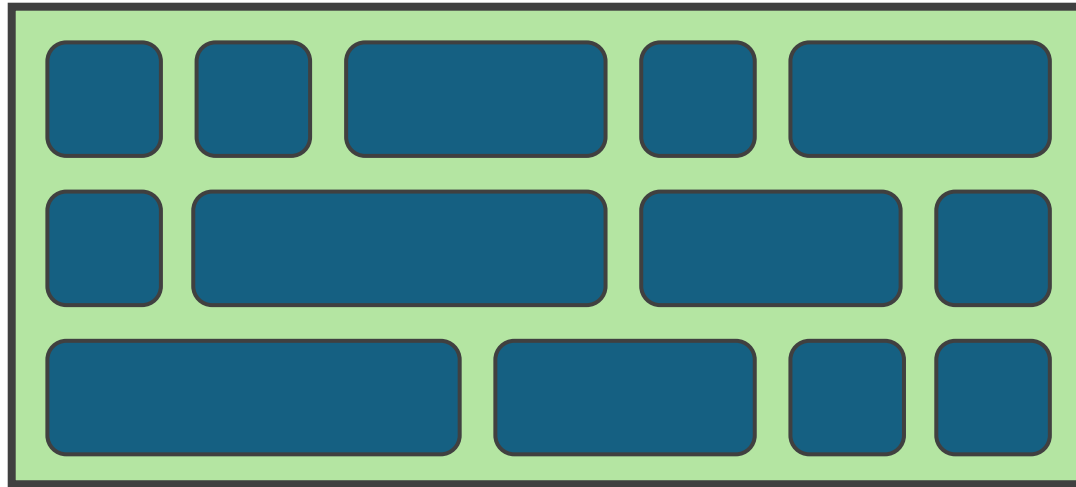
Euh...

Comment on sait si
un objet est toujours
utilisé ou non ?

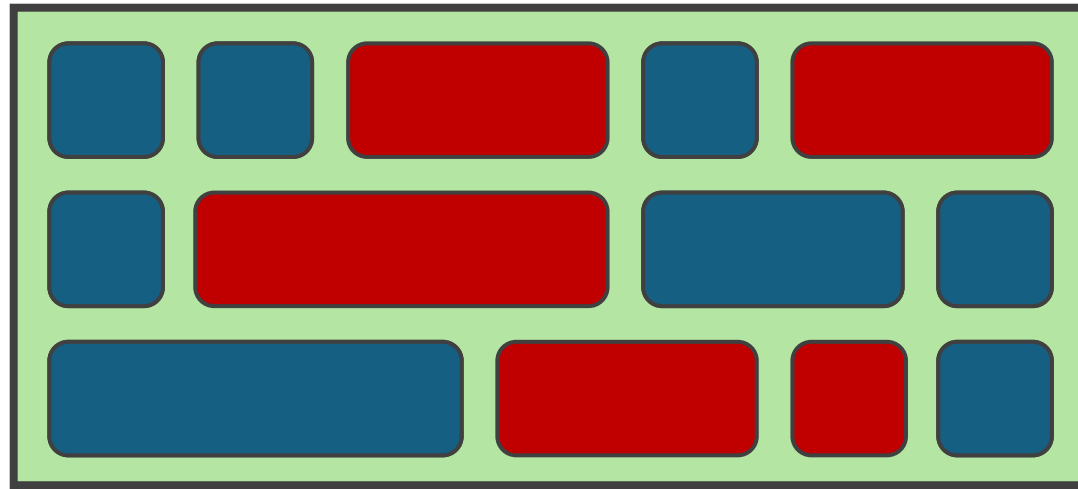
Savoir si un objet est vivant

- On parcourt l'arbre des objets en les marquant
- On part des GC roots
 - Stacks
 - Attributs static des classes
- Tout ce qui est marqué est utilisé
- Tout ce qui n'est pas marqué n'est plus utilisé

Java 1

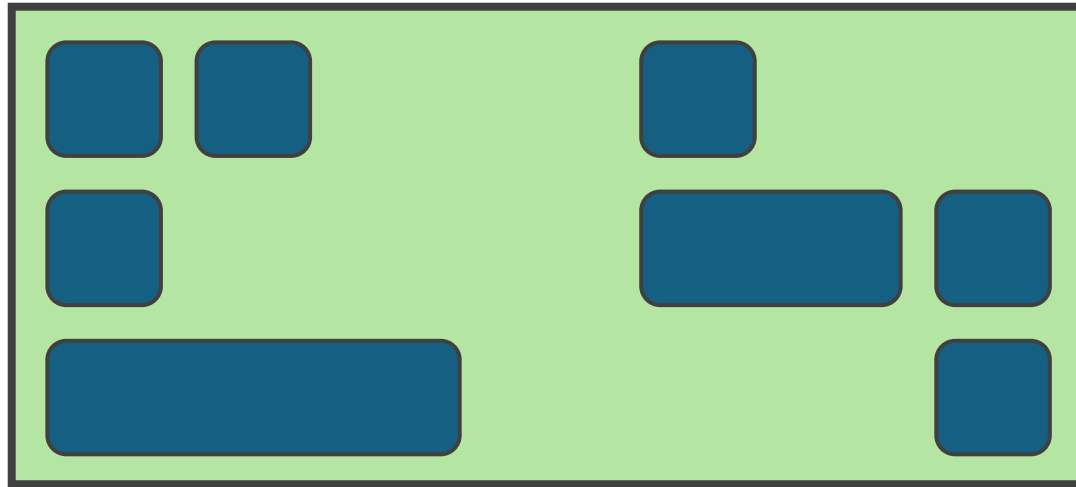


Java 1



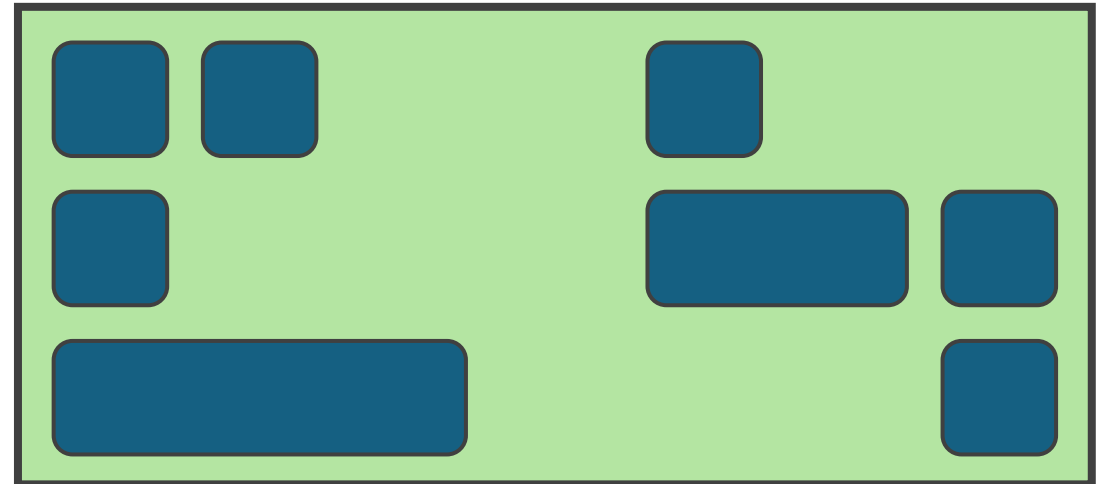
(fini)

Java 1



Java 1

- Ca marche et c'est déjà pas mal
- Mais c'est un peu lent...
- ...et ça fragmente la mémoire



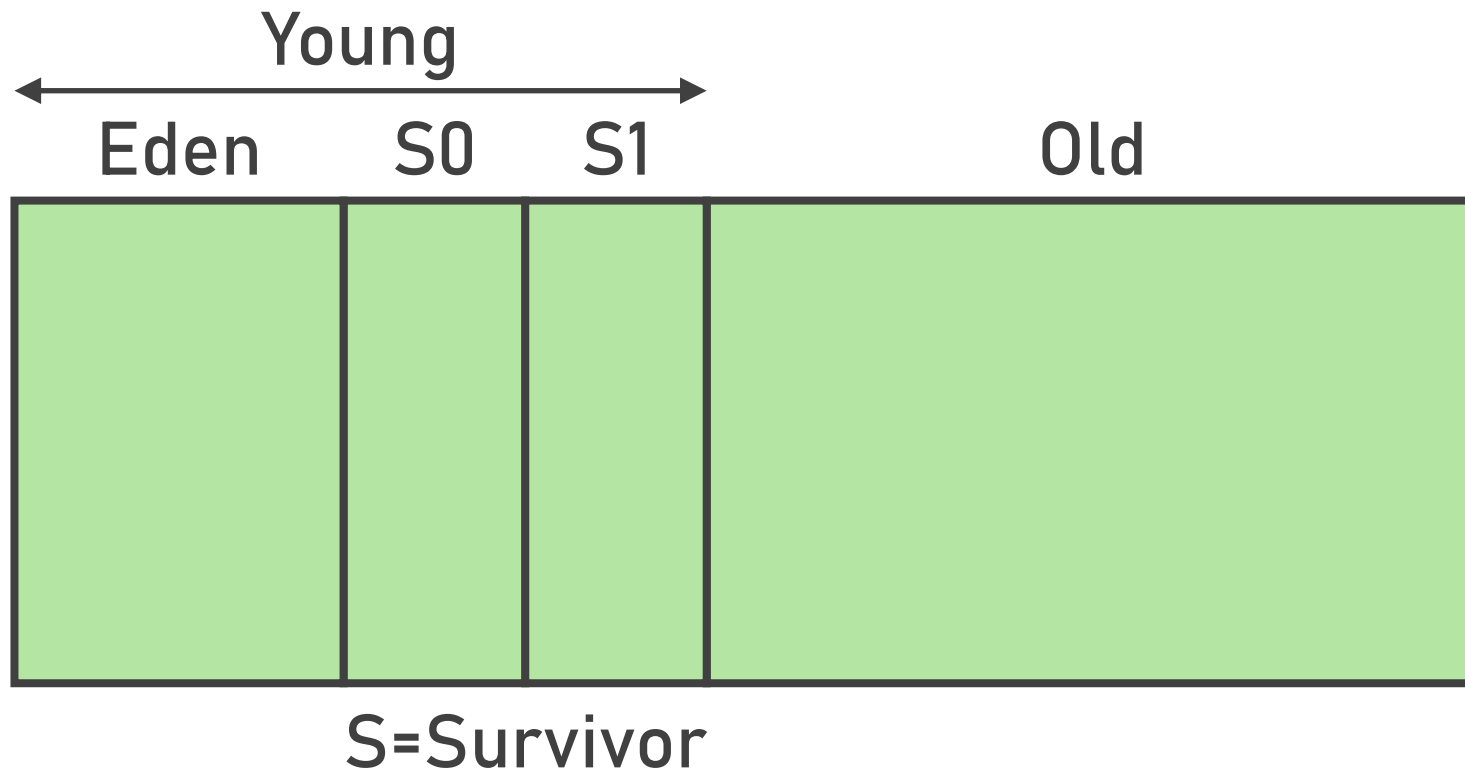
Comment aller plus vite ?

Ce serait bien d'éviter
de parcourir tous les
objets à chaque fois...

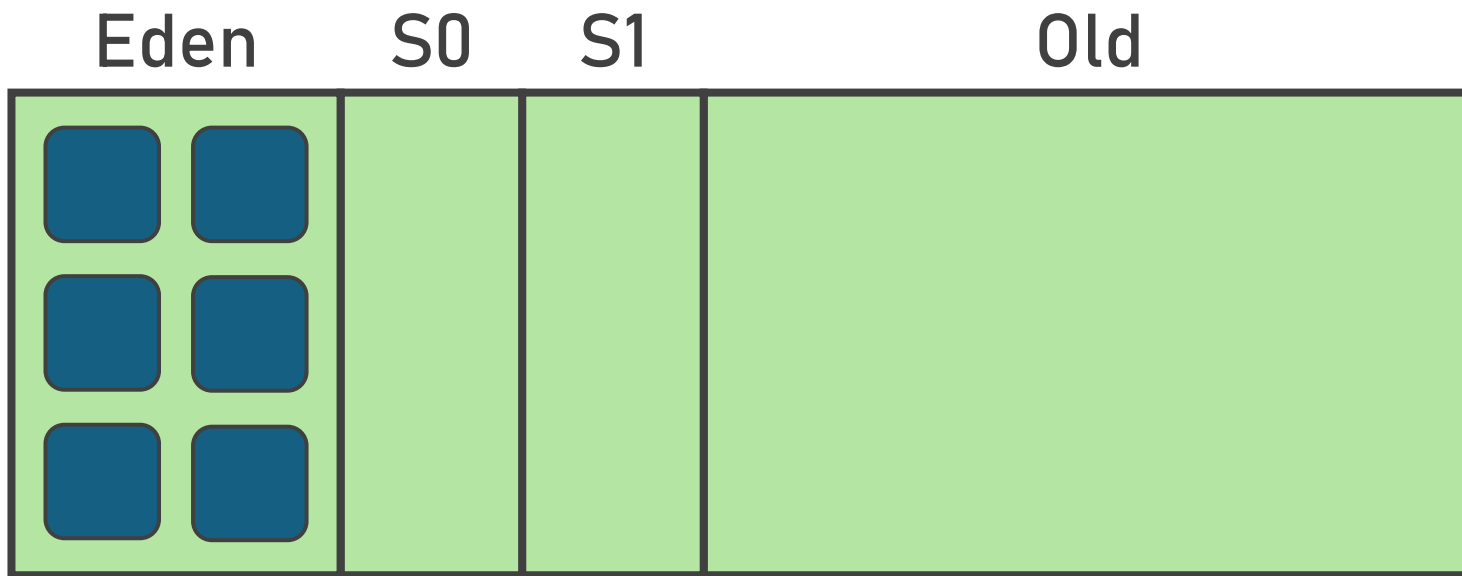
L'hypothèse générationnelle

- Soit les objets meurent rapidement
- Soit ils sont là pour longtemps (voire toujours)
- Java 1.2 (1998)
- Serial GC

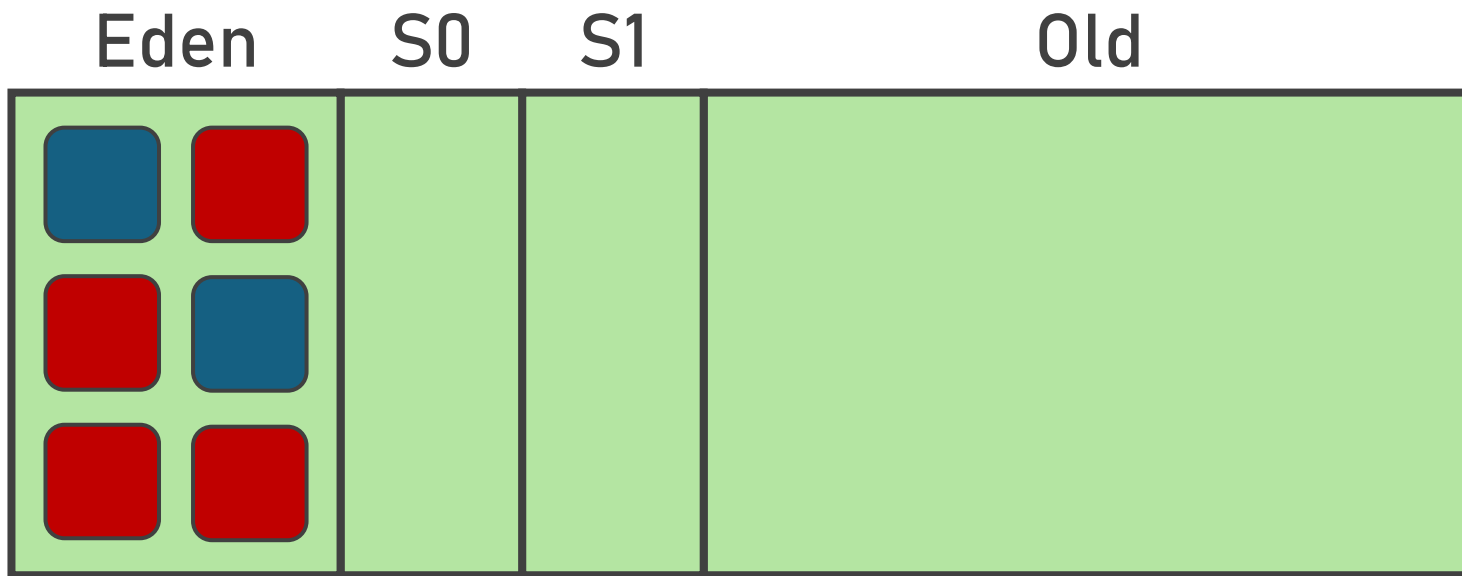
Serial GC



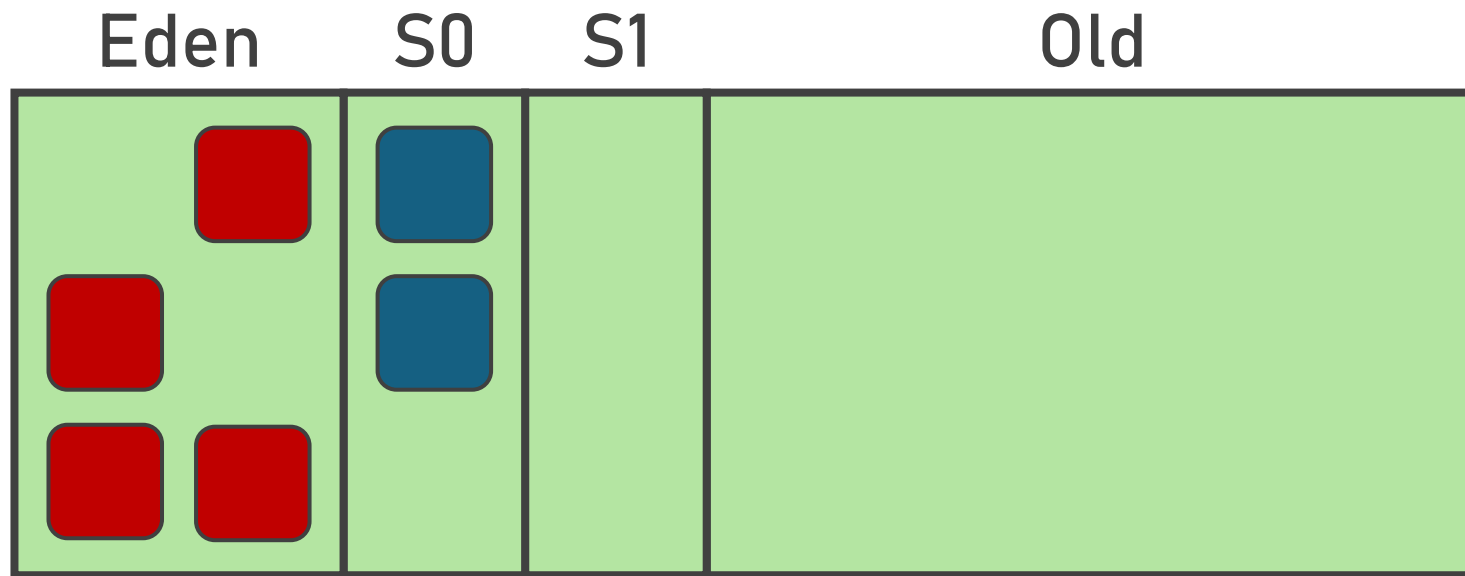
Serial GC



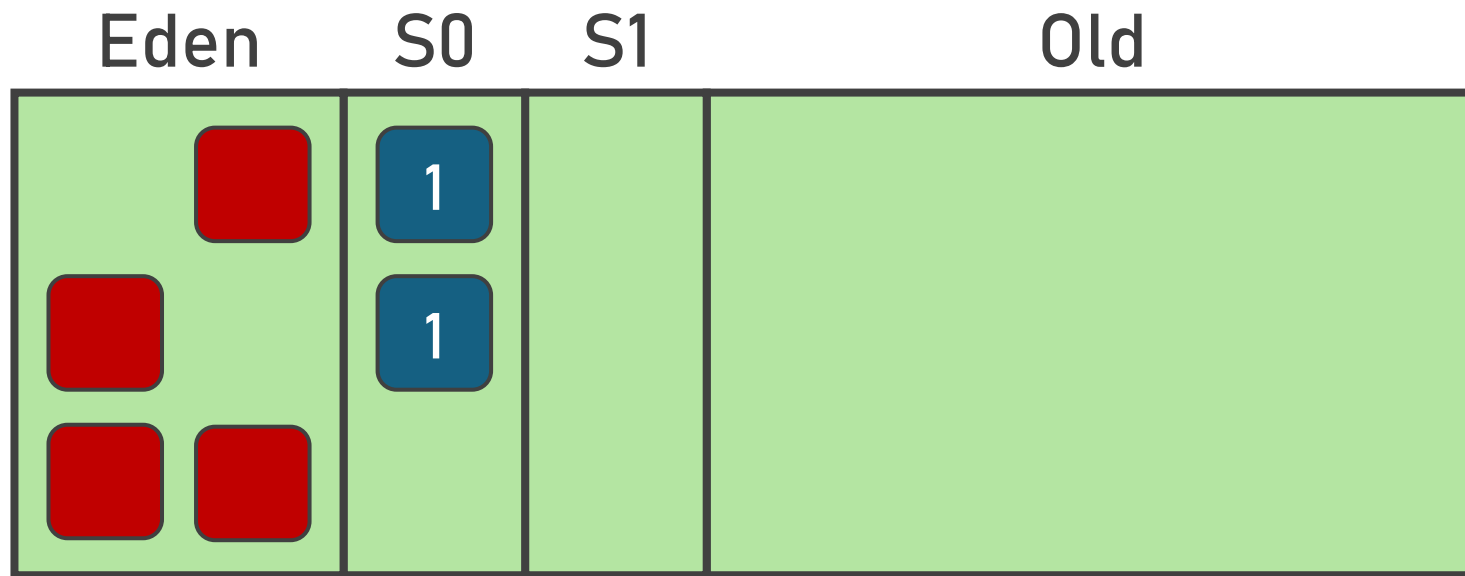
Serial GC



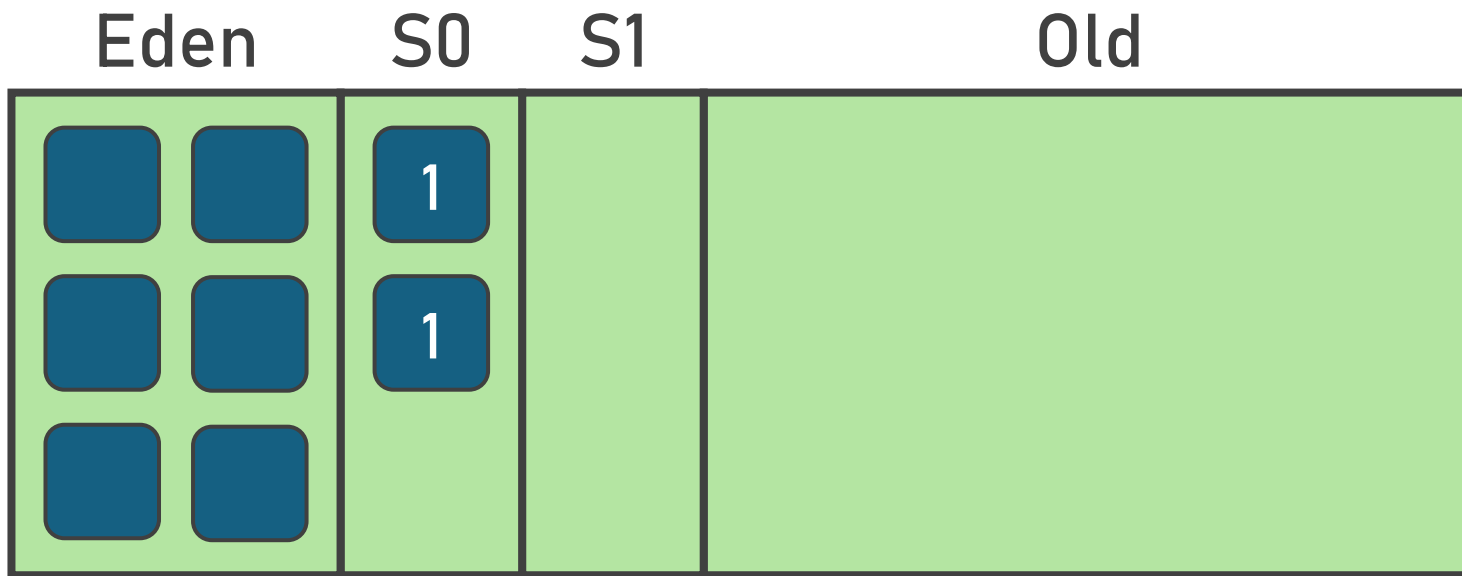
Serial GC



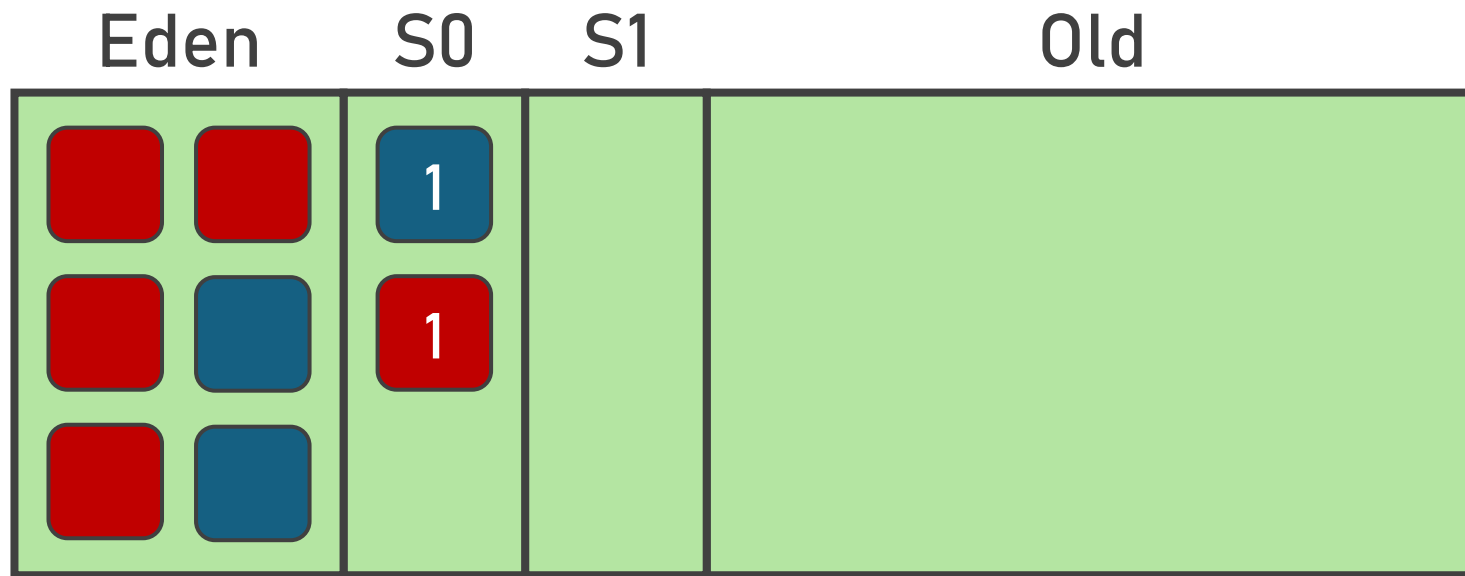
Serial GC



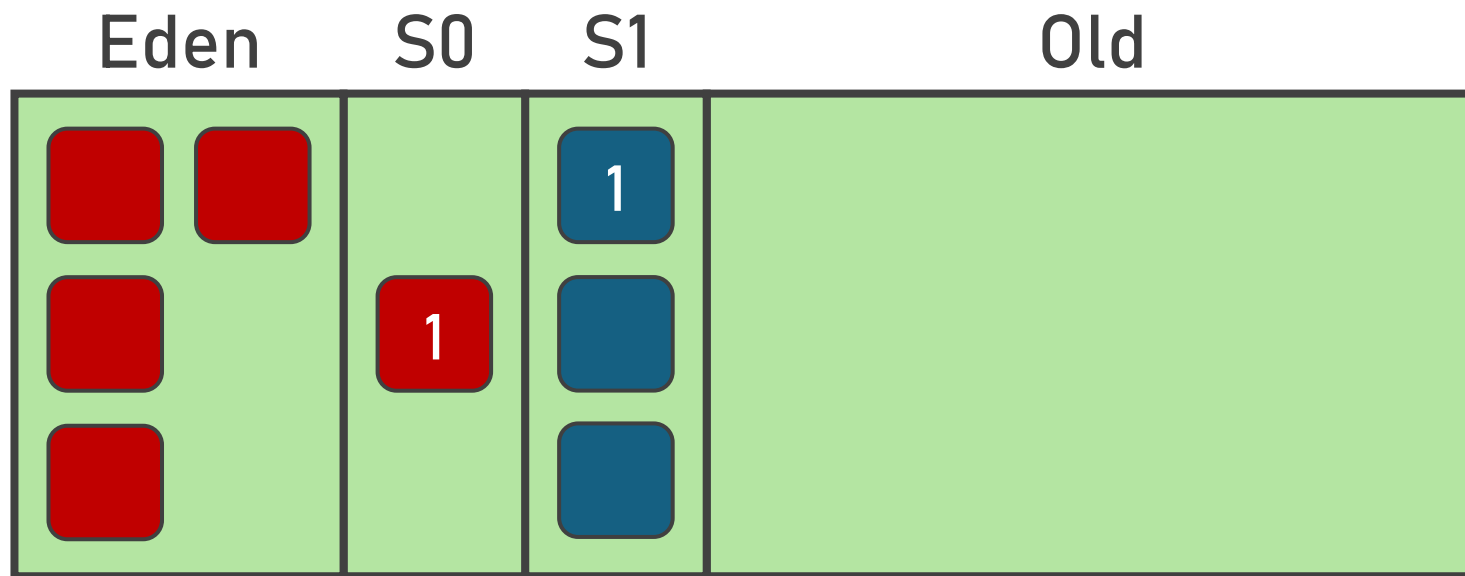
Serial GC



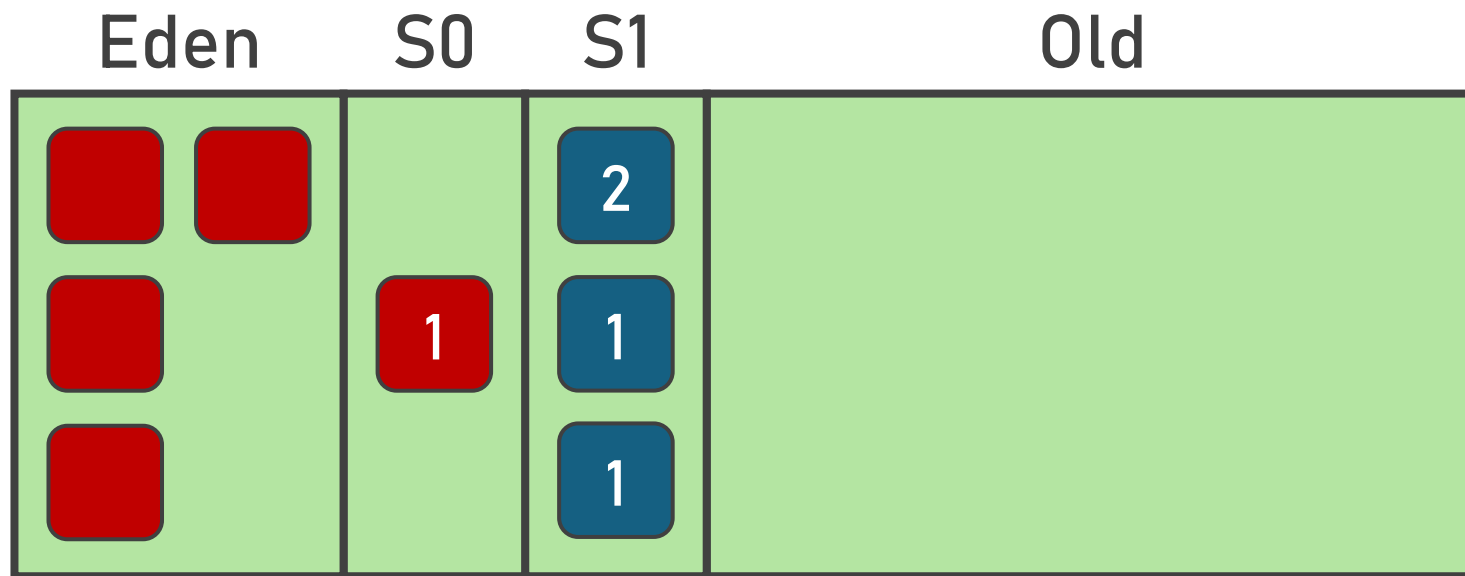
Serial GC



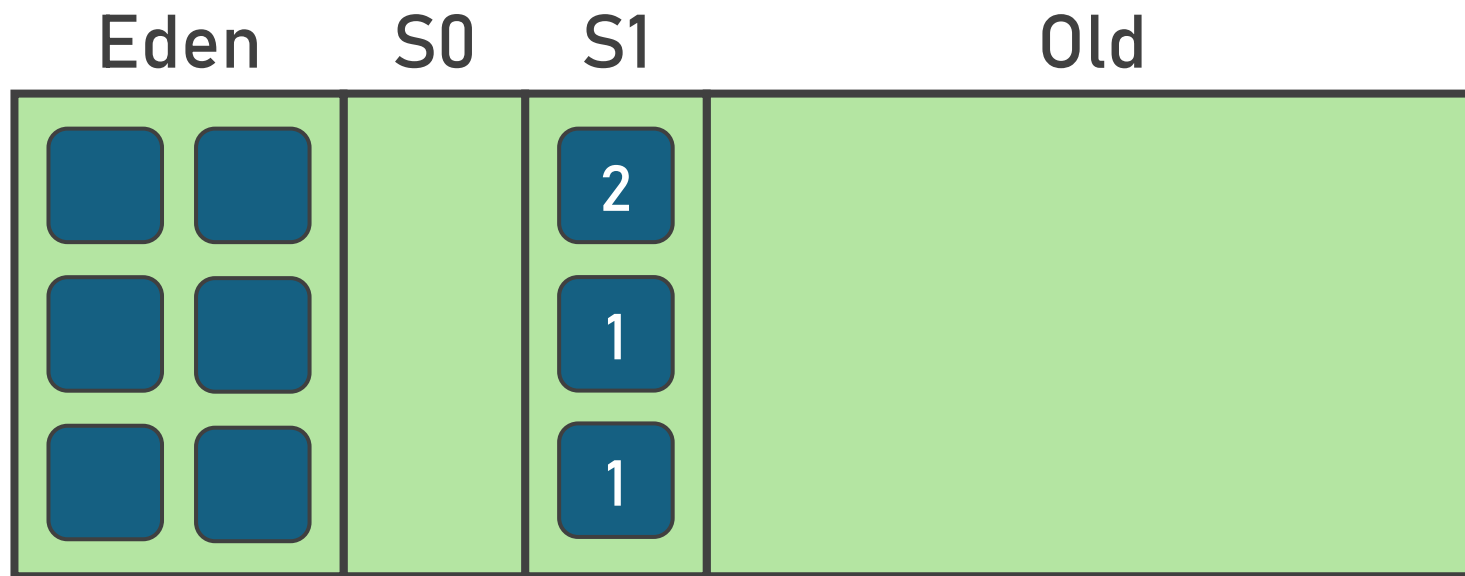
Serial GC



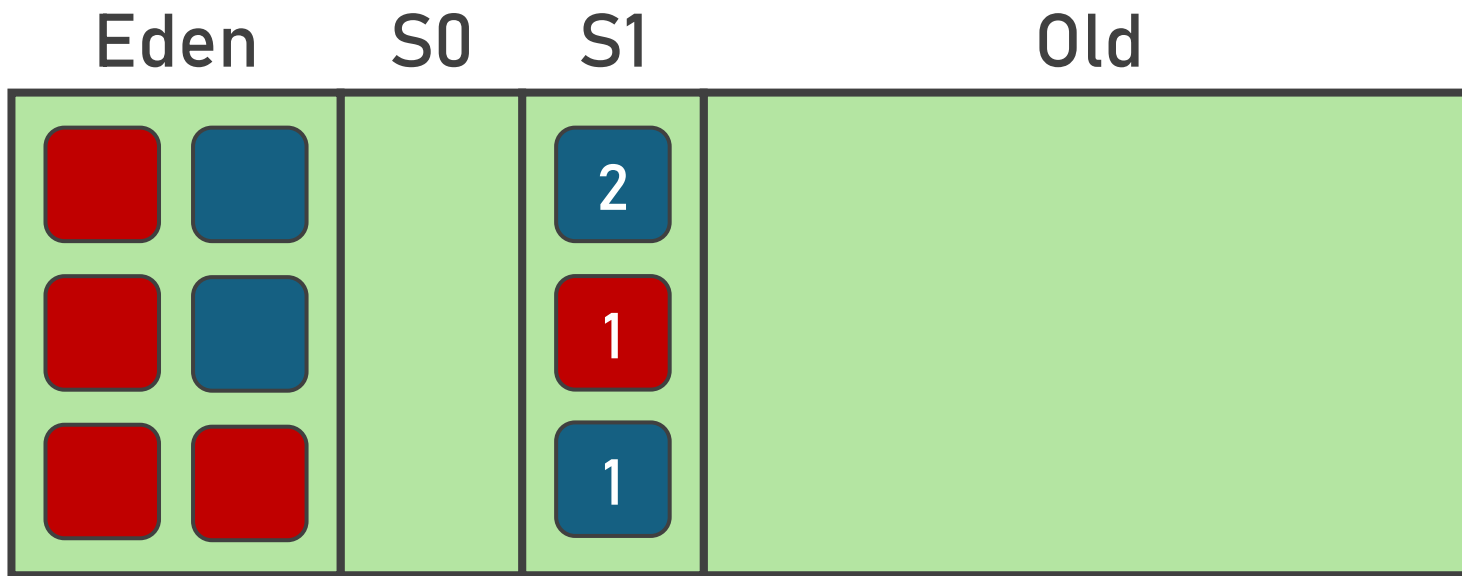
Serial GC



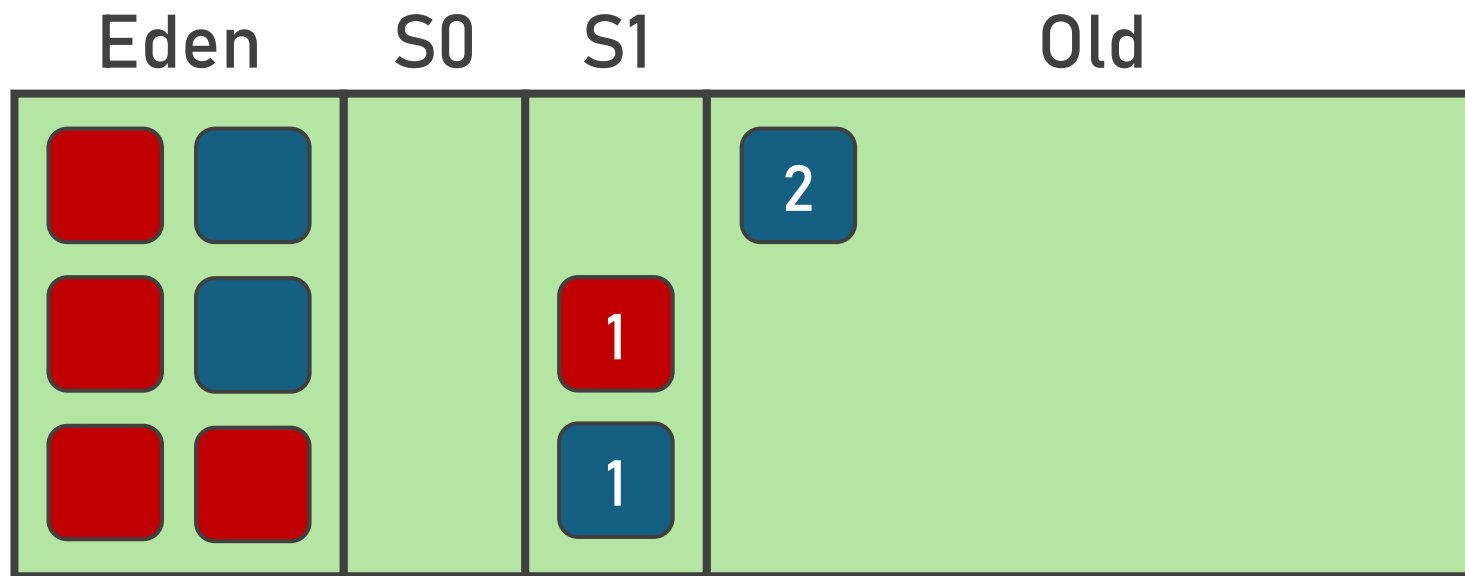
Serial GC



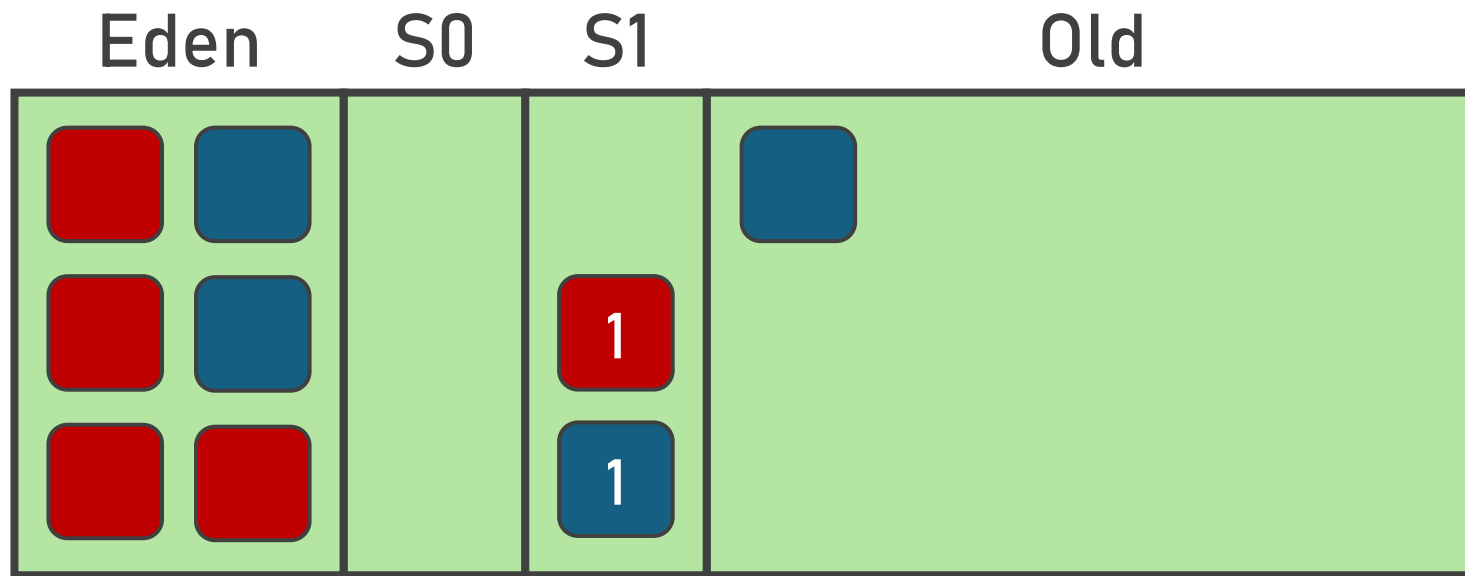
Serial GC



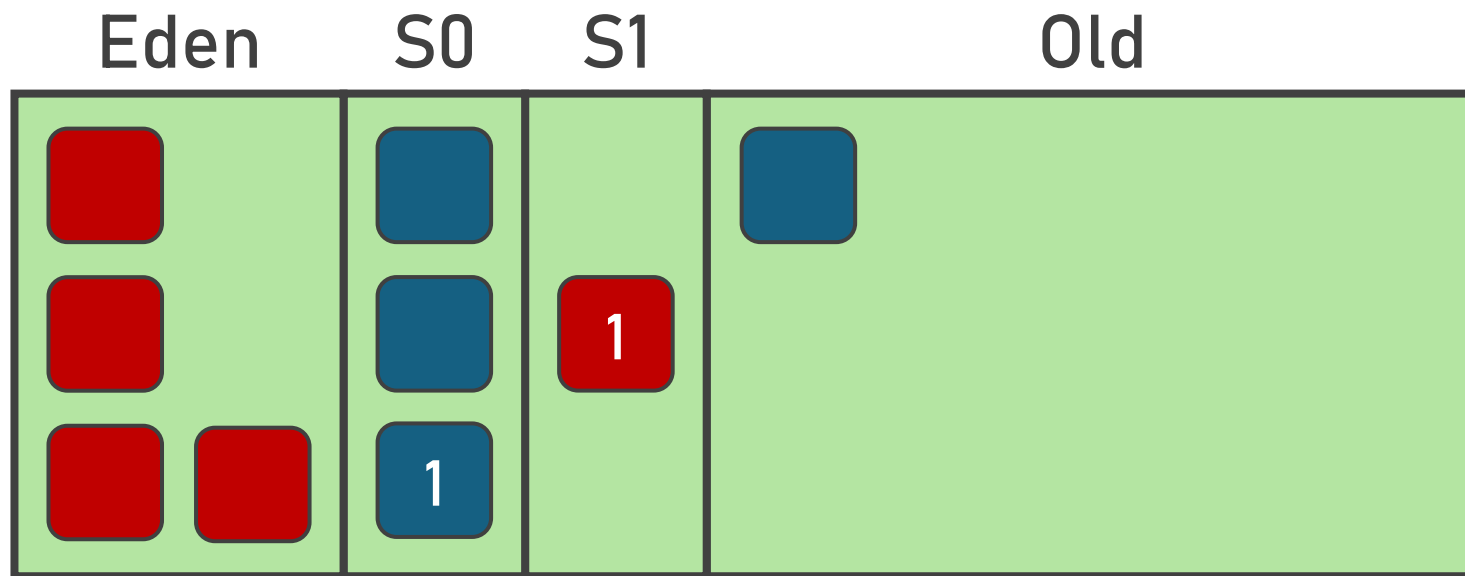
Serial GC



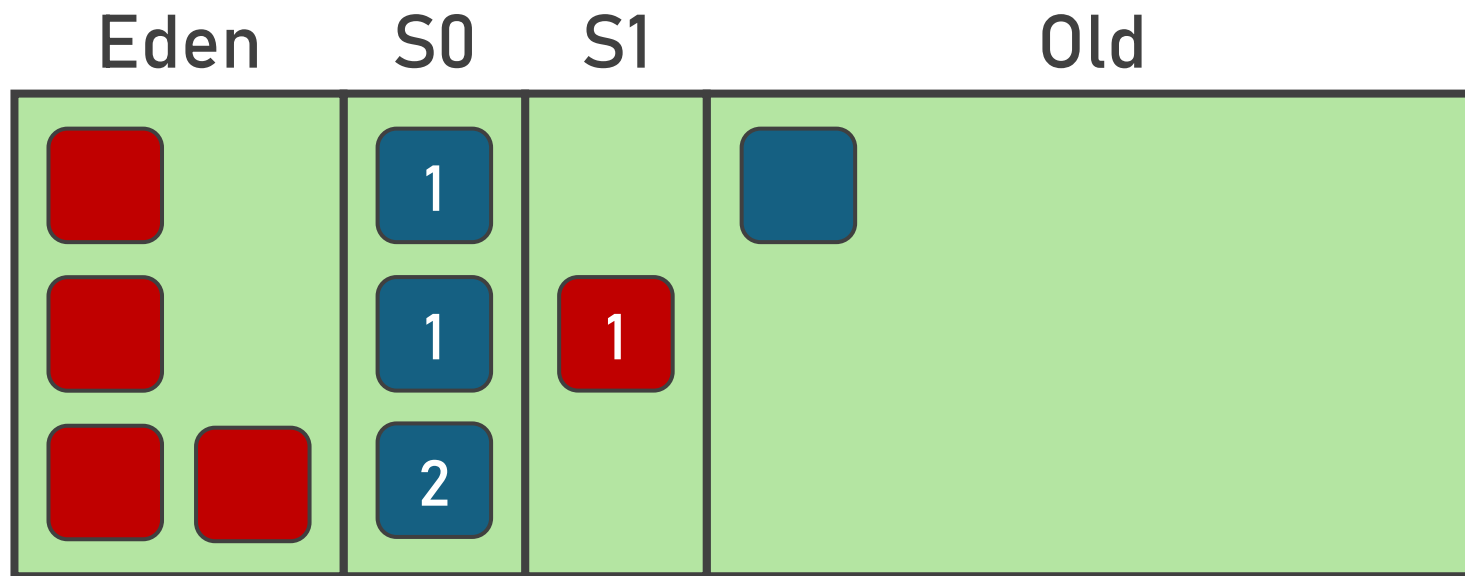
Serial GC



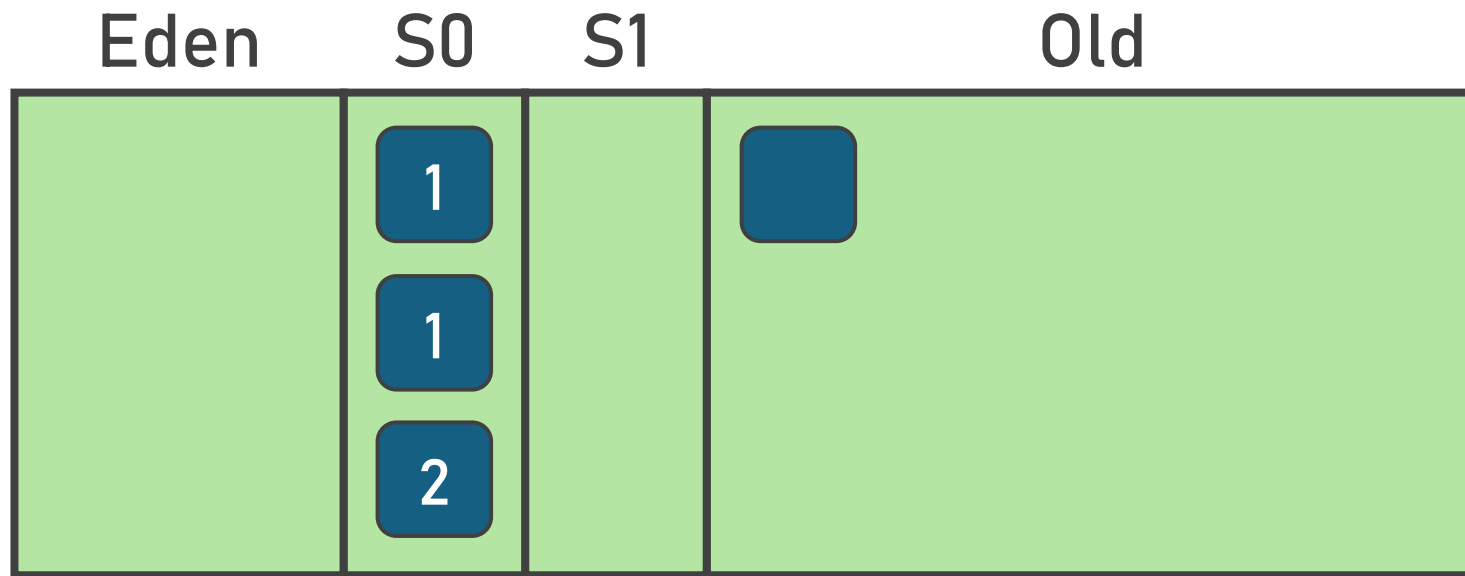
Serial GC



Serial GC

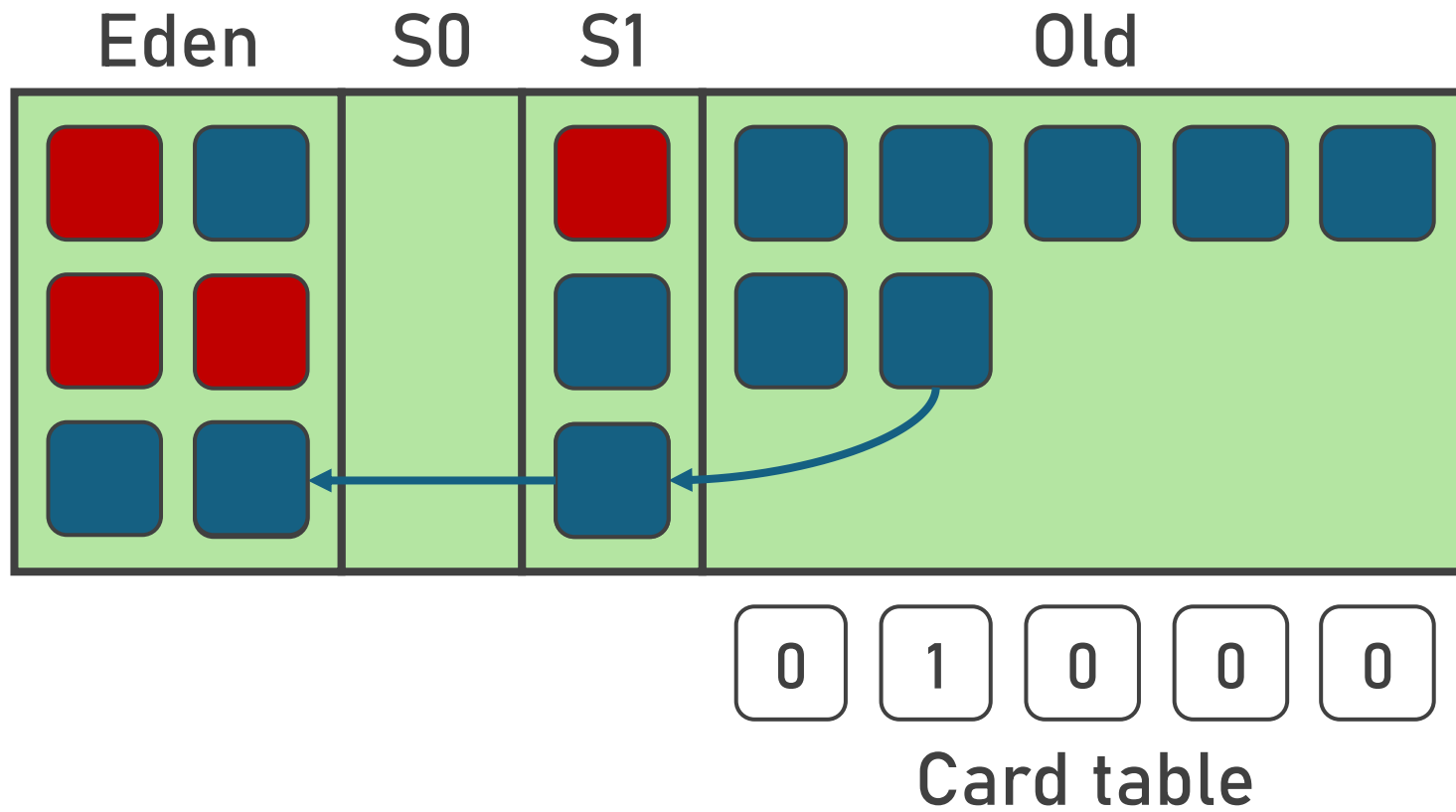


Serial GC

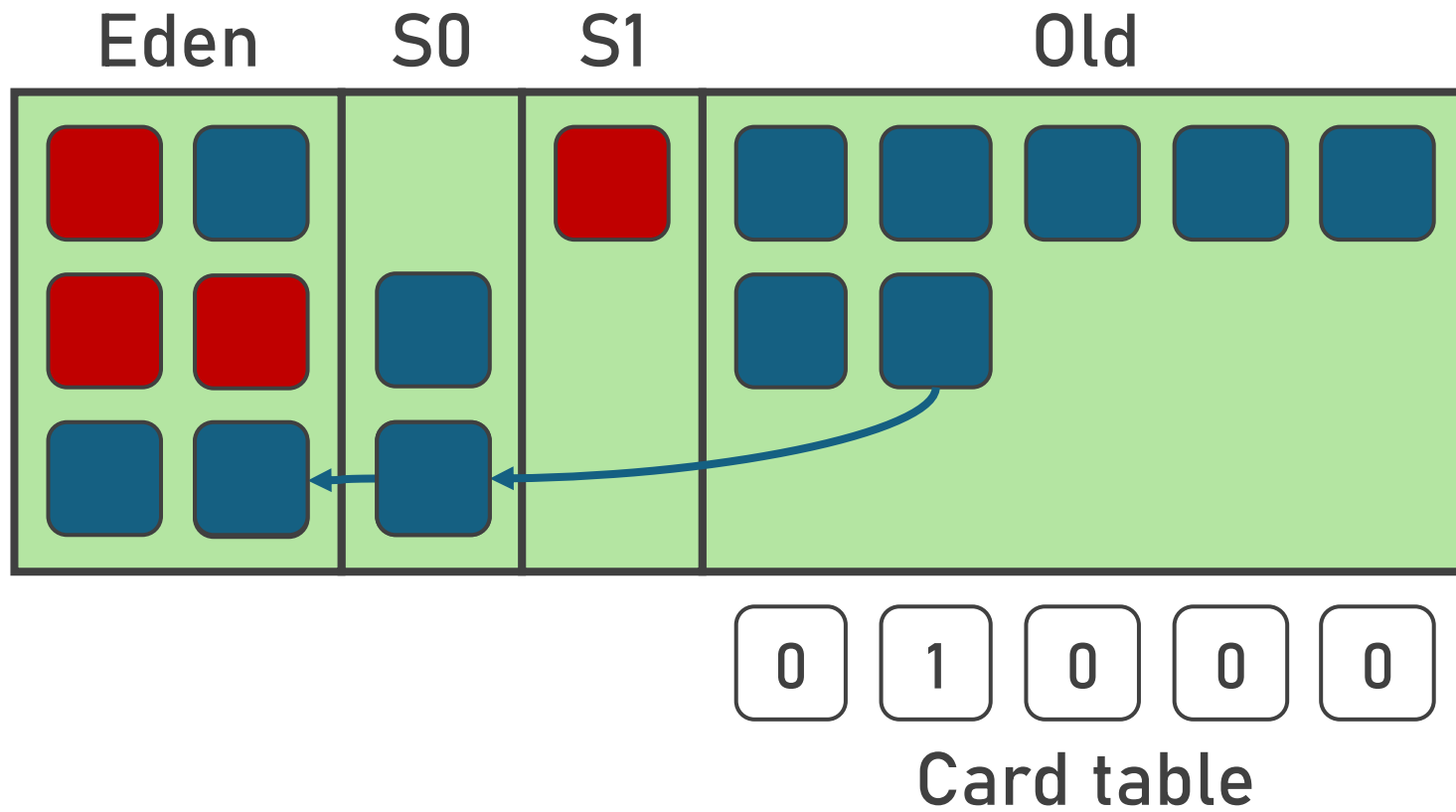


(et ça continue)

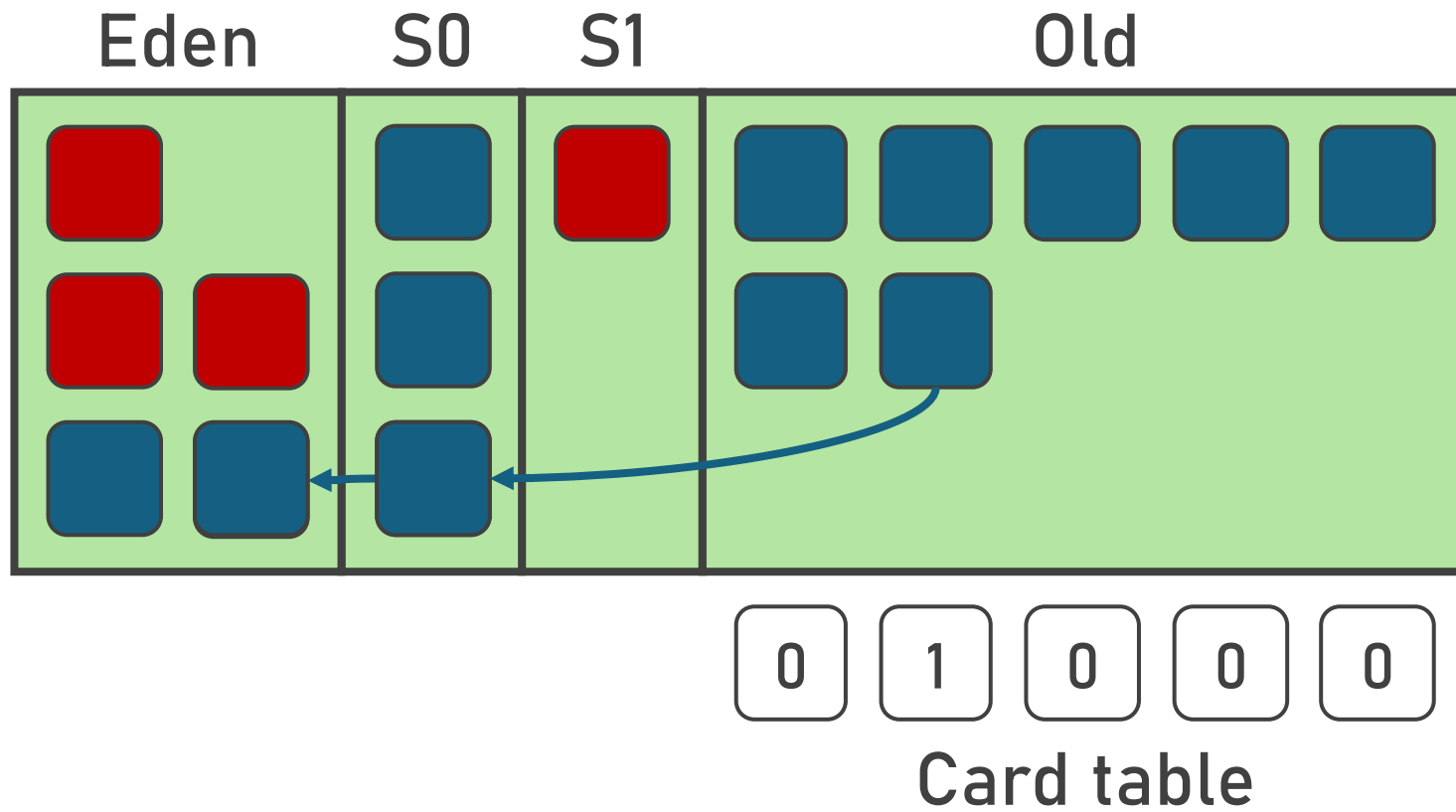
Serial GC



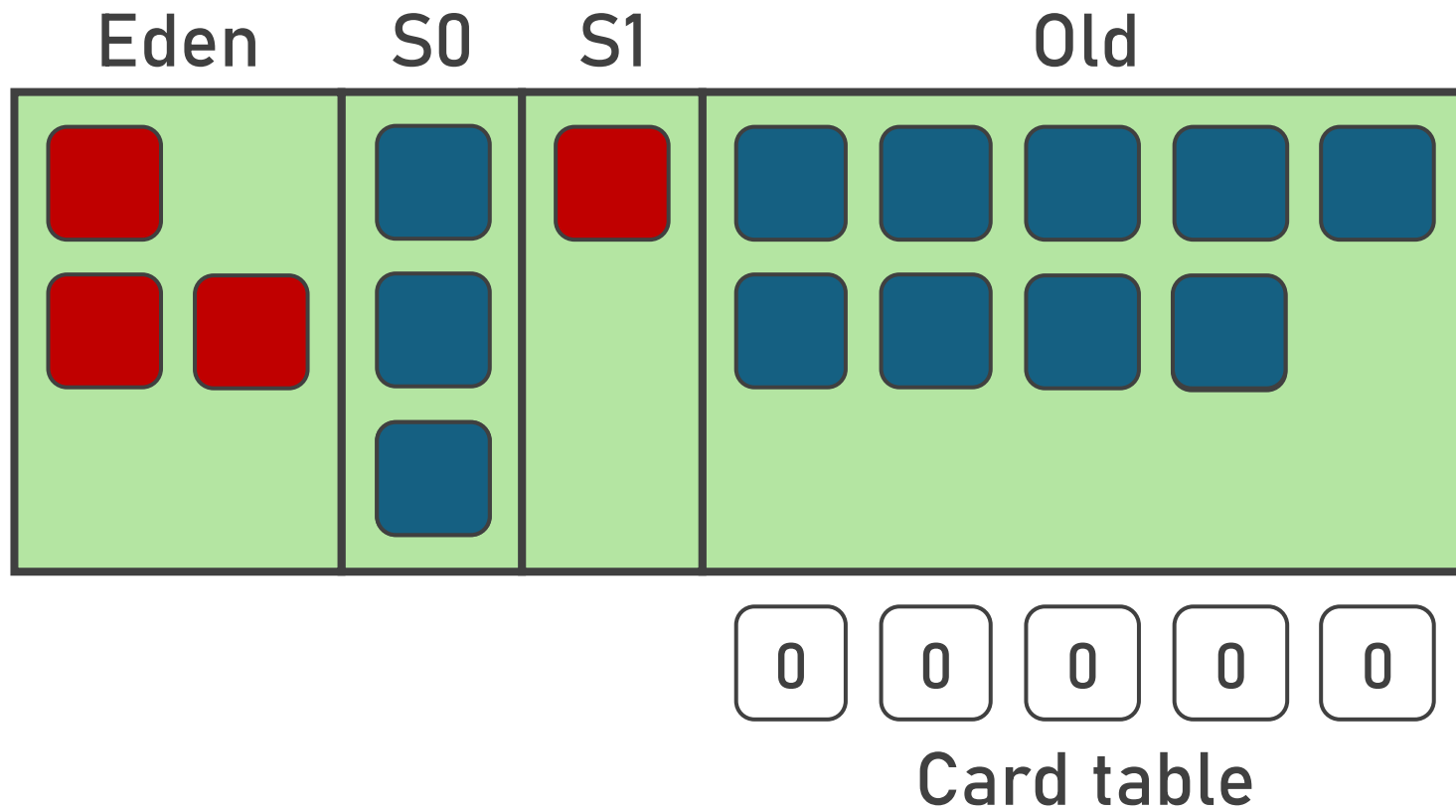
Serial GC



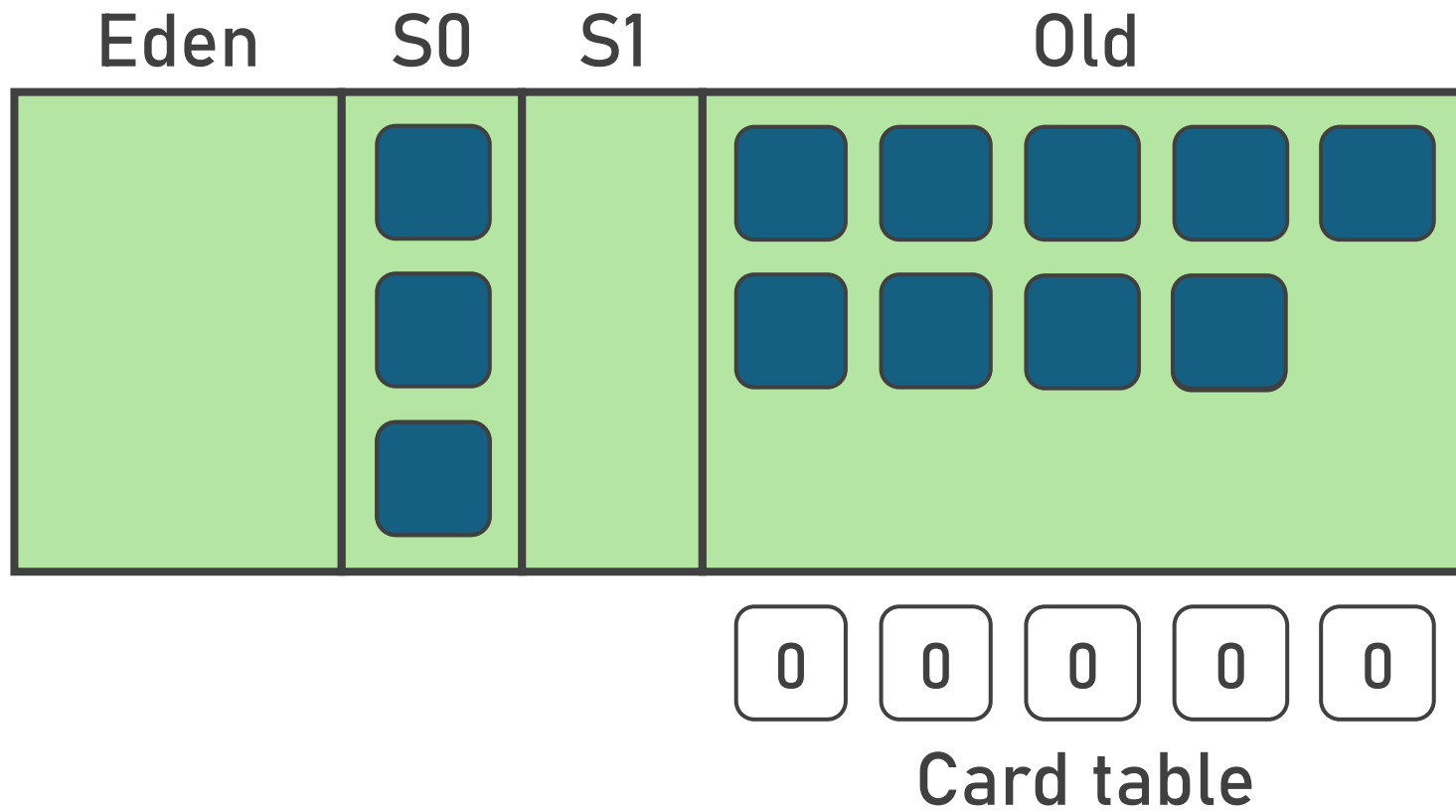
Serial GC



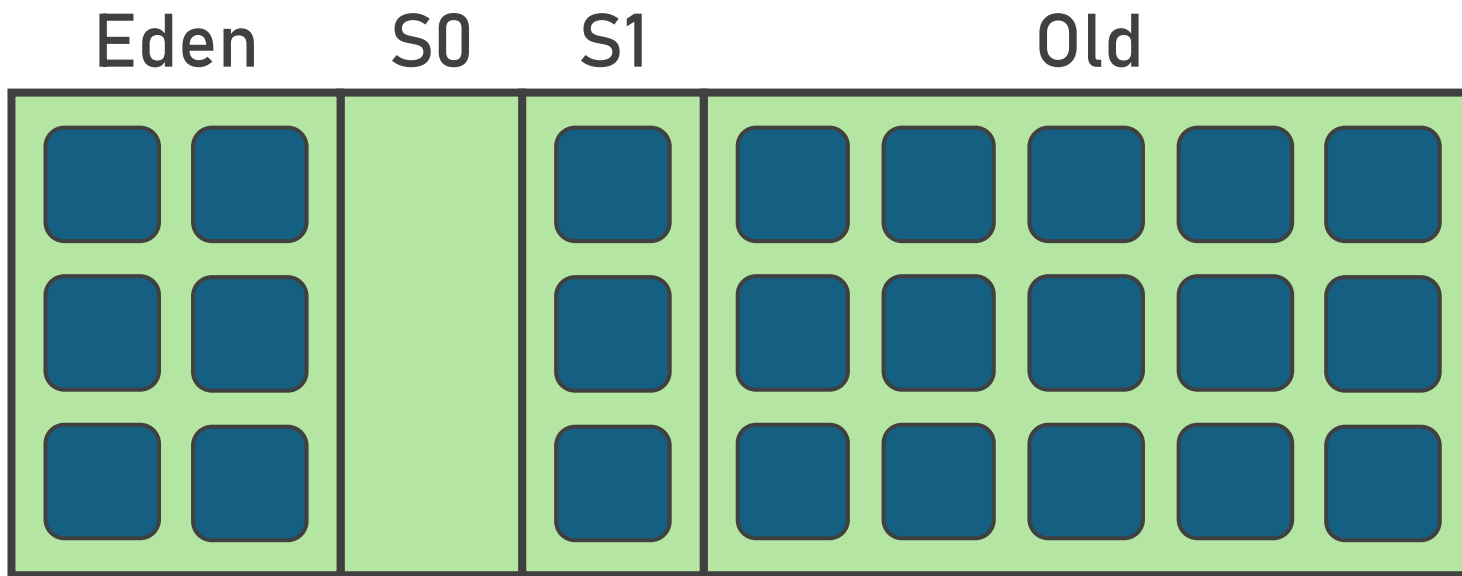
Serial GC



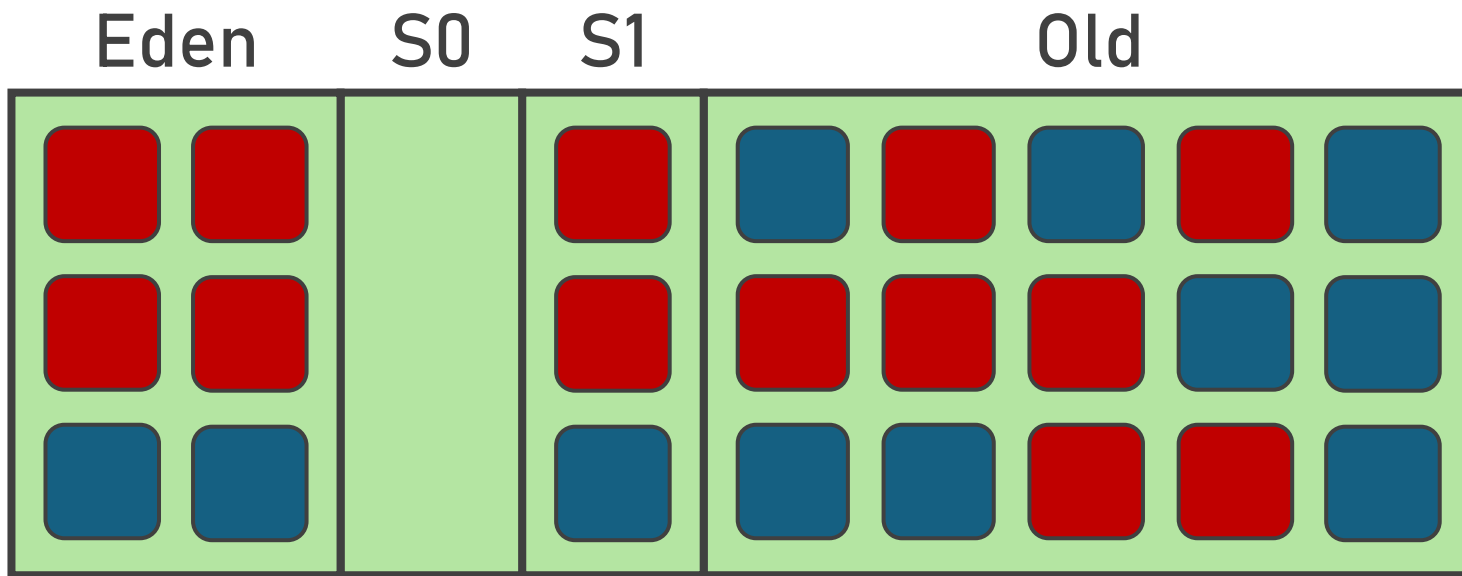
Serial GC



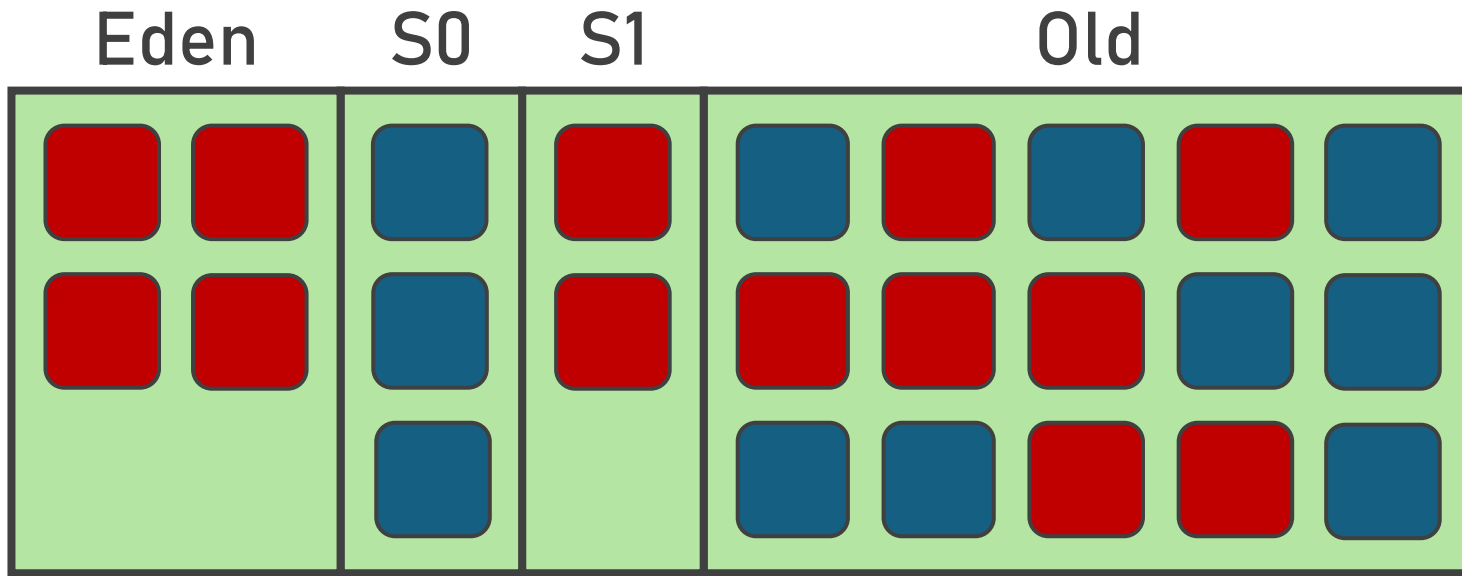
Serial GC



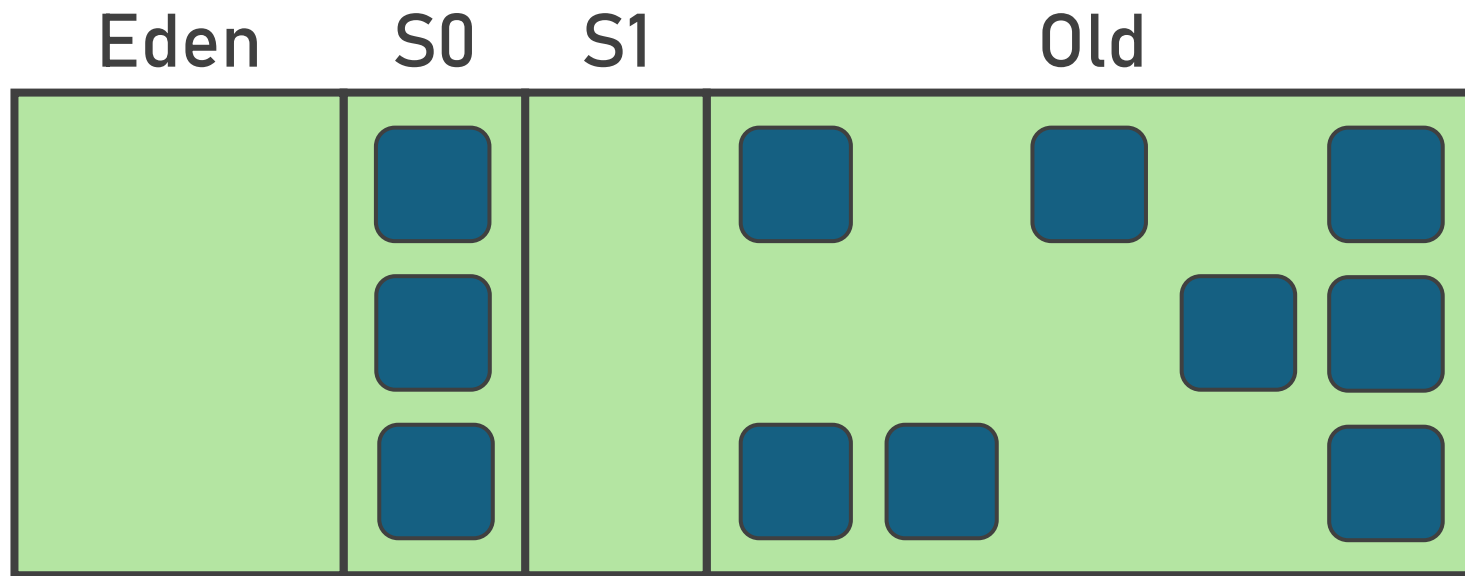
Serial GC – Full GC



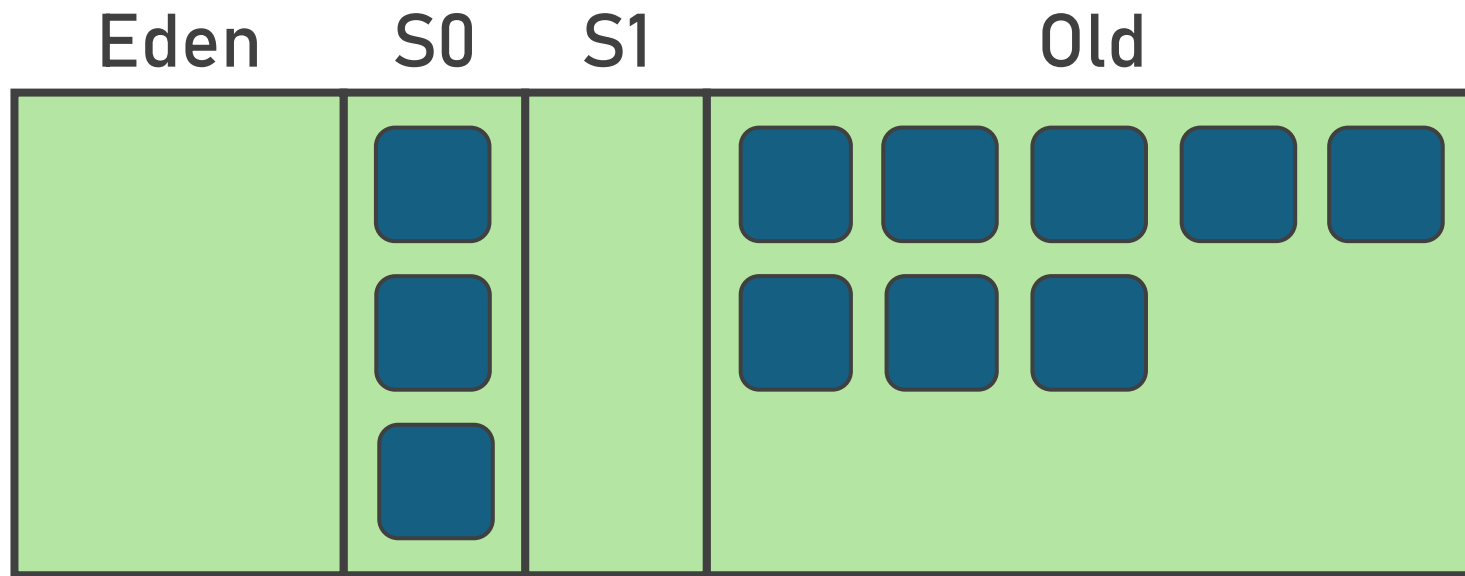
Serial GC – Full GC



Serial GC – Full GC



Serial GC – Full GC



Serial GC

- Il est toujours présent et maintenu
- Stop-the-world
- Un seul thread qui travaille
- Le GC avec le moins d'overhead
- Par défaut si < 2 Go de RAM ou < 2 CPUs
- Idéal pour les microservices

Parallel GC

- Java 5 (2004)
- La même chose mais en parallèle
- Plus d'overhead (synchronisation)

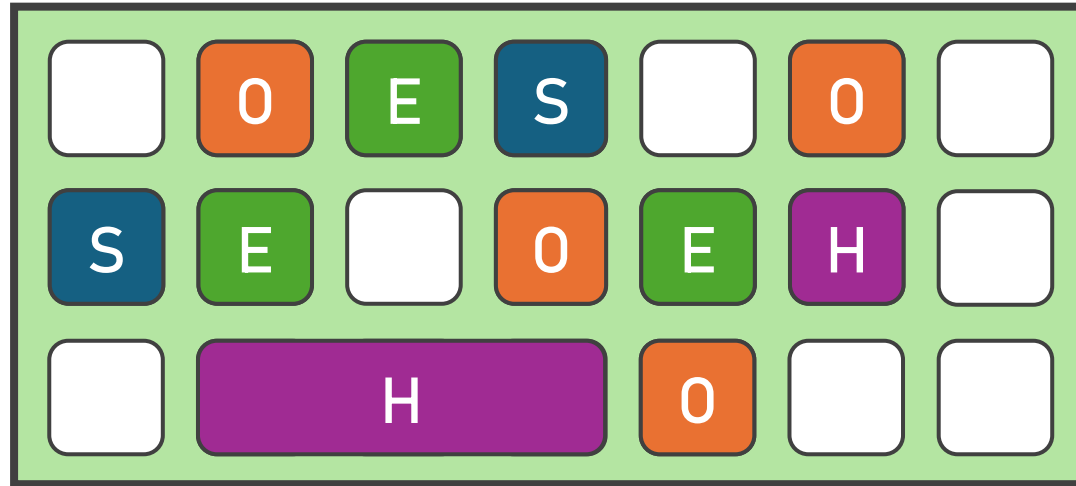
Parallel GC

- Le GC avec le plus de throughput
- Toujours Stop-The-World
- Jusqu'à plusieurs secondes sur grosses heaps
- Idéal pour le traitement de données, batchs, ...

Et la latence ?

- Le GC peut bloquer l'application plusieurs secondes
- Essai raté: Train GC
- Essai semi-raté: CMS GC
- Essai réussi : G1 GC en Java 7
 - Par défaut à partir de Java 9

G1 GC



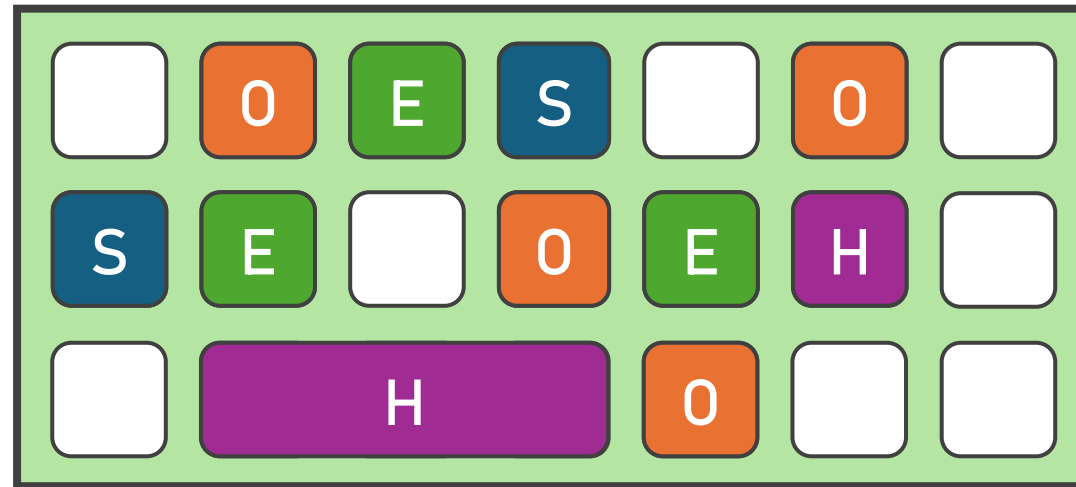
E Eden

S Survivor

O Old

H Humongous

G1 GC – Young collection



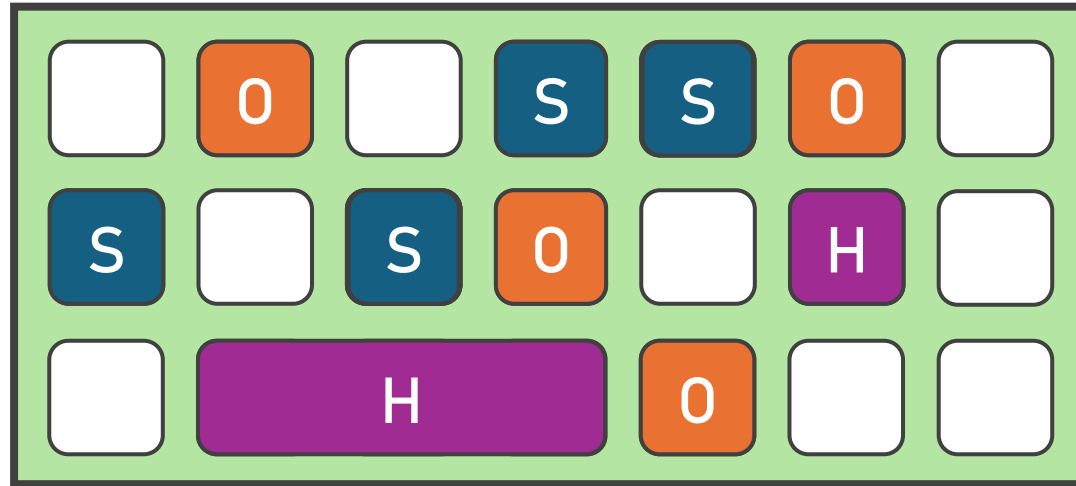
E Eden

S Survivor

O Old

H Humongous

G1 GC – Young collection



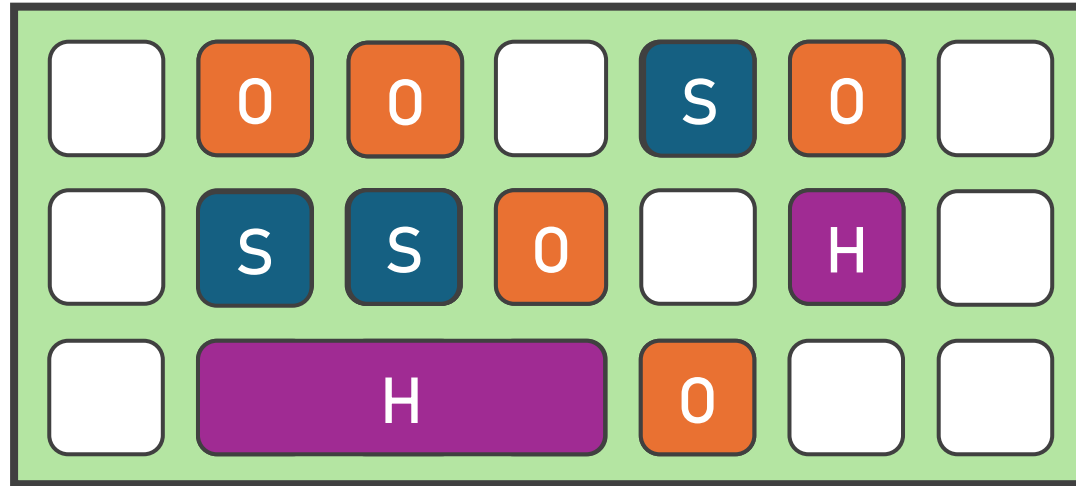
E Eden

S Survivor

0 Old

H Humongous

G1 GC – Young collection



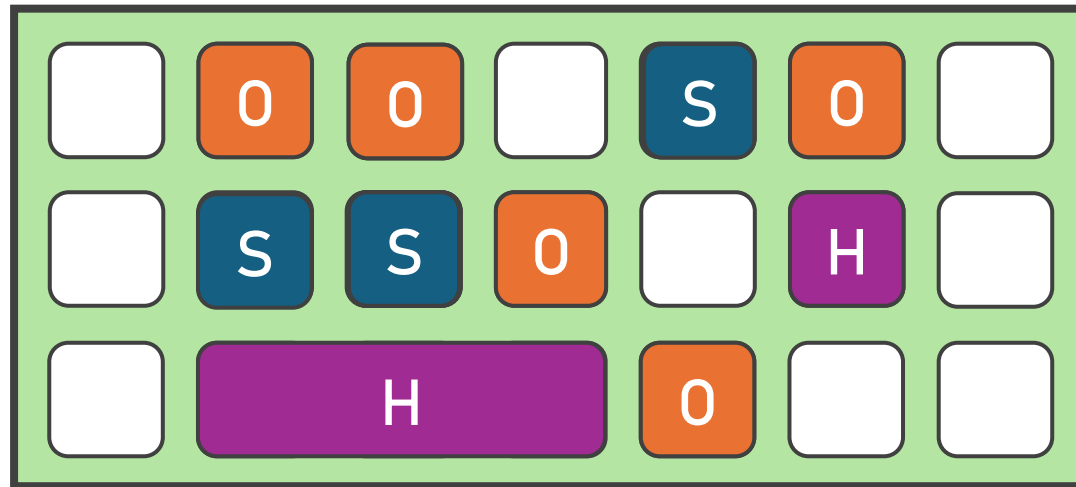
E Eden

S Survivor

O Old

H Humongous

G1 GC – Mixed collection



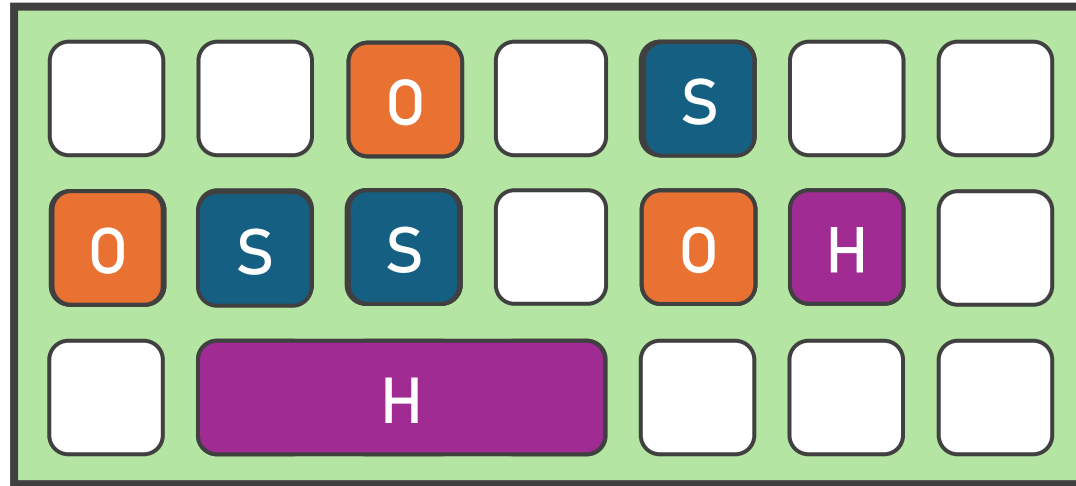
E Eden

S Survivor

0 Old

H Humongous

G1 GC – Mixed collection



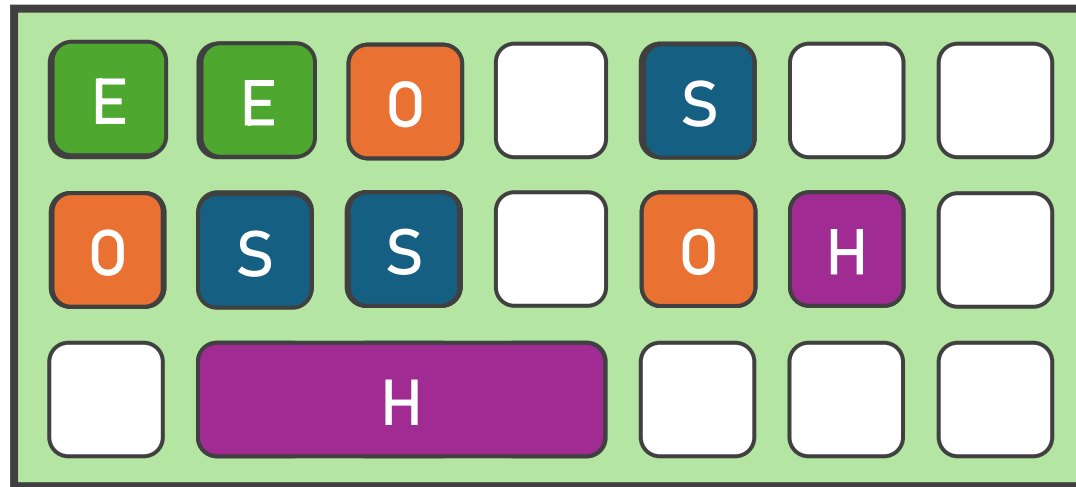
E Eden

S Survivor

O Old

H Humongous

G1 GC



E Eden

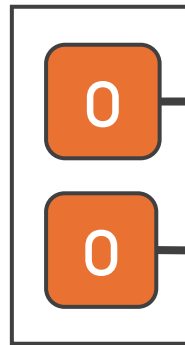
S Survivor

O Old

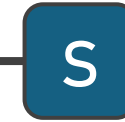
H Humongous

G1 GC – Région

RememberSet



Old



G1 GC

- Il commence par les régions avec le plus de déchets
- Il copie les objets vivants dans d'autres régions
- Partiellement Stop-The-World
- Il s'arrête après 200ms
- Le GC « généraliste »

Et la latence ?

- Pause maximum: 200ms *



- Il peut faire un Full GC
 - Il n'y a pas d'espace continu pour un Humongous
 - Il ne peut pas déplacer de la young à la old car la old est trop fragmentée
 - Il n'y a plus de région vide

Et la latence ?

- Pause maximum: 200ms **



- C'est par JVM
- En micro-services, on peut enchaîner les GCs sur les différentes JVMs...
 - C'est la poisse mais ça arrive

Ultra low latency

- Pauses en dessous de la milliseconde
- Tout est fait en concurrence
 - Overhead important (mémoire + CPU)
- ZGC et Shenandoah GC
- Générationnel avec régions

Idée: stocker des informations dans les pointeurs

- Adresse: jusqu'à 16To
- Flag finalizer
- Remapped: l'adresse est bonne
- Marks: il faut aller récupérer la bonne adresse



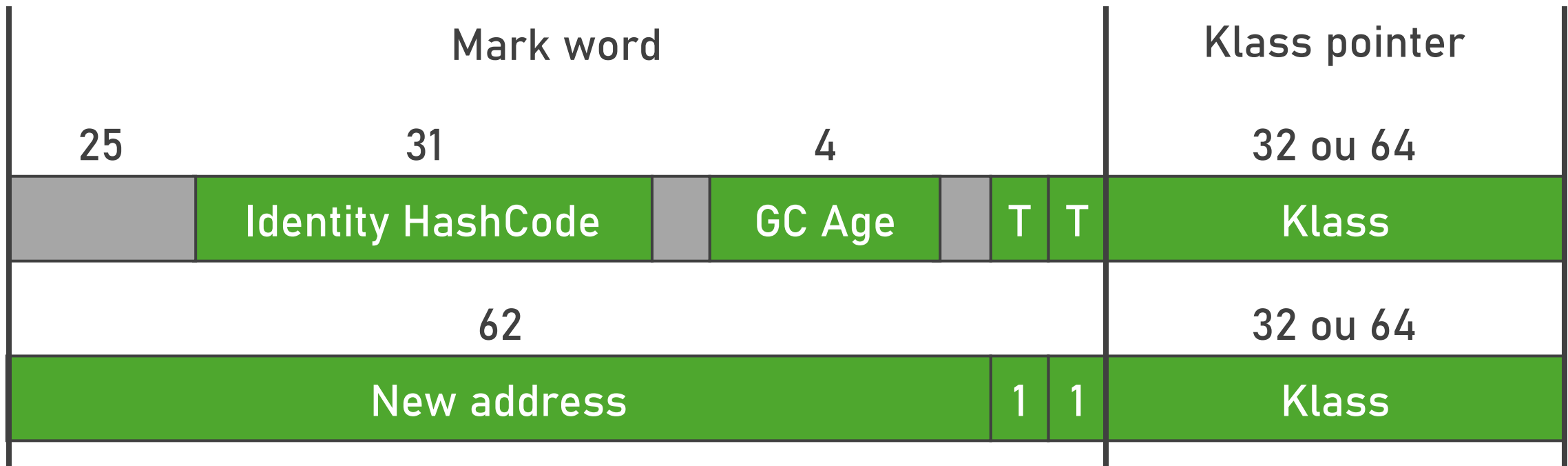
ZGC

- Auto-tune
- Le thread peut mettre à jour sa référence au besoin
- Inconvénients:
 - Pointeurs de 64 octets
 - Allocation stalls

Shenandoah GC

Idée : stocker des informations dans l'entête de l'objet

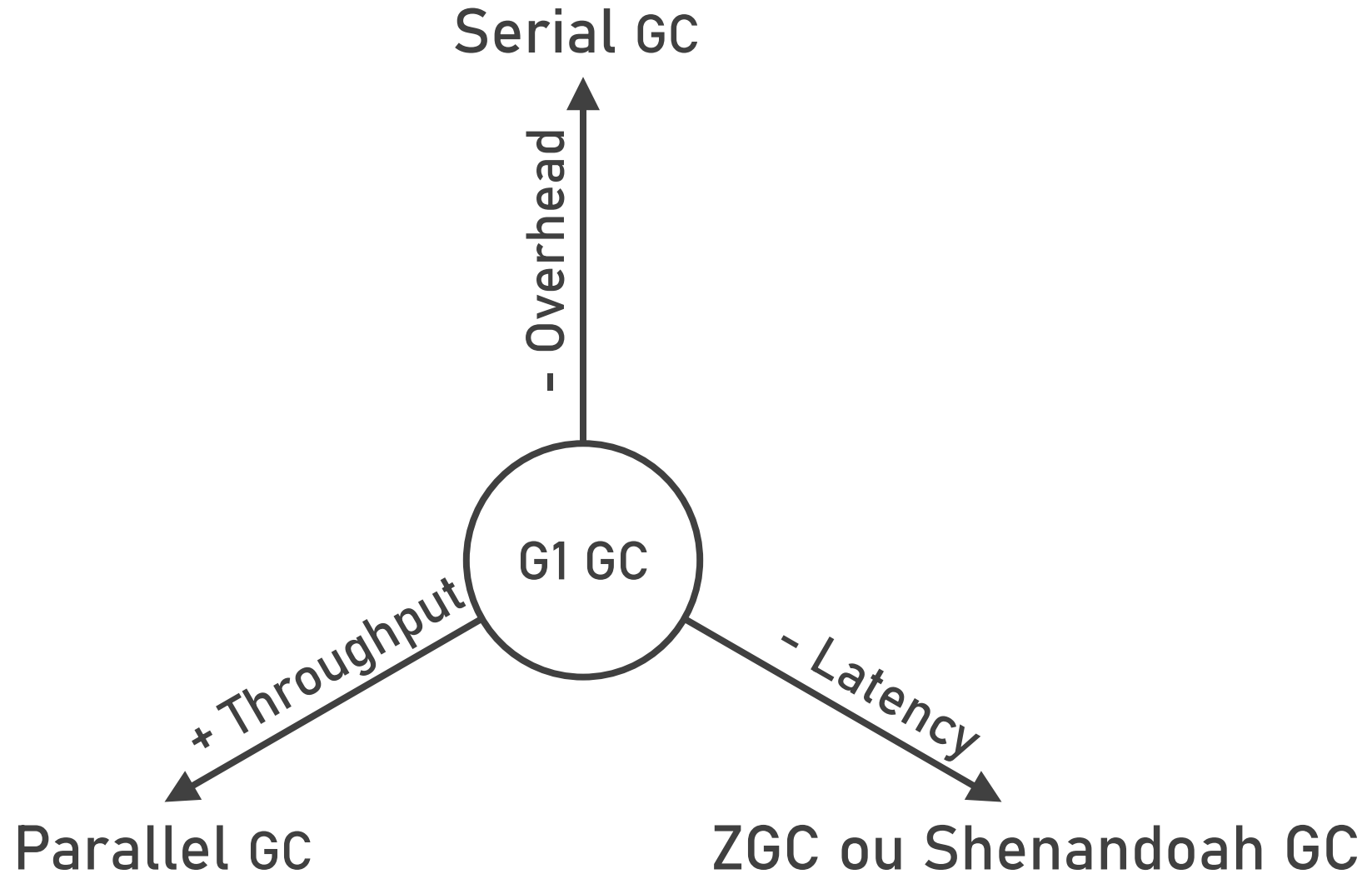
- Tags: 01=unlocked, 00/10=locked, 11=GC



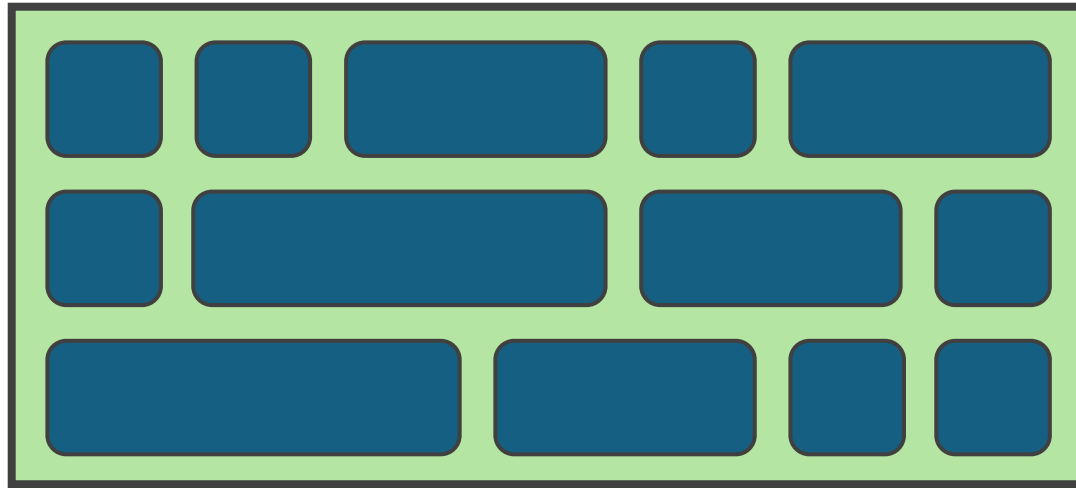
Shenandoah GC

- Très configurable
- Inconvénients:
 - Mise à jour de toutes les référence avant réécriture
 - Allocation stalls voire Full GC

Quel GC choisir



Epsilon GC



Epsilon GC



Epsilon GC

- Aucun nettoyage, crash si toute la mémoire est prise
- Utile les processus à très courte durée de vie
- Utile pour serverless
- Utile pour les micro-benchmarks

Java en natif ?

- Avec Quarkus
 - Serial GC
 - Epsilon GC
- Avec Graal VM
 - Serial GC
 - Epsilon GC
 - G1 GC (version enterprise seulement)

Pour améliorer les perfs

- Choisir le GC adapté à son besoin
- Mesurer
 - Avoir des logs du GC
 - Ne jamais prendre la latence moyenne mais les percentiles
- Avoir une JVM récente

Flags magiques

- `-Xlog:gc*:file=gc.log:time,uptime,mem`
- `-Xms = -Xmx`
- `-XX:+UseLargePages`
- `-XX:+AlwaysPreTouch`

Le futur

- Amélioration du throughput de G1 (JEP-522, JDK 26)
 - 5-15% d'amélioration
- G1 GC remplace Serial GC sur les petites JVMs (JEP-523)
 - Car le throughput est proche et la latence bien meilleure
- Bootstrap plus rapide de ZGC (prévu)
- Dimensionnement automatique de la Heap (prévu)
 - G1 + Serial GC

Pour aller plus loin

HotSpot Virtual Machine Garbage Collection Tuning Guide

Java / Java SE / 25

HotSpot Virtual Machine Garbage Collection Tuning Guide

1 Introduction to Garbage Collection Tuning

A wide variety of applications, from small applets on desktops to web services on large servers, use the Java Platform, Standard Edition (Java SE). In support of this diverse range of deployments, the Java HotSpot VM provides multiple garbage collectors, each designed to satisfy different requirements. Java SE selects the most appropriate garbage collector based on the class of the computer on which the application is run. However, this selection may not be optimal for every application. Users, developers, and administrators with strict performance goals or other requirements may need to explicitly select the garbage collector and tune certain parameters to achieve the desired level of performance. This document provides information to help with these tasks.

First, general features of a garbage collector and basic tuning options are described in the context of the serial, stop-the-world collector. Then specific features of the other collectors are presented along with factors to consider when selecting a collector.

Topics

- [What Is a Garbage Collector?](#)
- [Why Does the Choice of Garbage Collector Matter?](#)
- [Supported Operating Systems in Documentation](#)

What Is a Garbage Collector?

The garbage collector (GC) automatically manages the application's dynamic memory allocation requests.

Pour aller plus loin

Le blog de Thomas SCHATZL

tschatzl @ github

About

Aug 12, 2025

[JDK 25 G1/Parallel/Serial GC changes](#)

JDK 25 RC 1 [is out](#), and as is custom by now, here is a brief overview of the changes to the stop-the-world collectors in OpenJDK for that release.

Apr 1, 2025

[JDK 24 G1/Parallel/Serial GC changes](#)

JDK 24 has been [released](#) a few weeks ago. This post provides a brief overview of the changes to the stop-the-world collectors in OpenJDK in that release. Originally I thought there was not much to talk about, hence the delay, but in hindsight I have been wrong.

Feb 21, 2025

[New Write Barriers for G1](#)

The Garbage First (G1) collector's throughput sometimes trails that of other HotSpot VM collectors - by up to 20% (e.g. [JDK-8253230](#) or [JDK-8132937](#)). The difference is caused by G1's principle to be a garbage collector that balances latency and throughput and tries to meet a pause time goal. A large part of that can be attributed to the synchronization of the garbage collector with the application necessary

Merci pour votre écoute

Antoine DESSAIGNE



Slides



Feedback

